

Sesión 2

GraphQL en MLOps + grafos de linaje



Consultar metadatos y relaciones con flexibilidad: cuando REST se queda corto.

Agenda de hoy

0 : 00



Recap y encuadre

Dónde estamos en la plataforma

0 : 15



Grafos y su importancia

Nodos, aristas y por qué importan en los datos

0 : 45



Bases de datos de grafos

Neo4j y Cypher · SQL vs grafo

1 : 20



Grafos de linaje en MLOps

Trazabilidad de datos → modelo → despliegue

1 : 35



GraphQL y sus ventajas vs REST

Esquema, queries, mutations · over/under-fetching

2 : 10



Práctica

Neo4j + Cypher · GraphQL con Strawberry · GraphQL sobre grafo

2 : 50



Mini-TP 2 y cierre

Metadatos de tu modelo por GraphQL

La plataforma ya habla REST; hoy sumamos consultas flexibles

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

En la Sesión 1

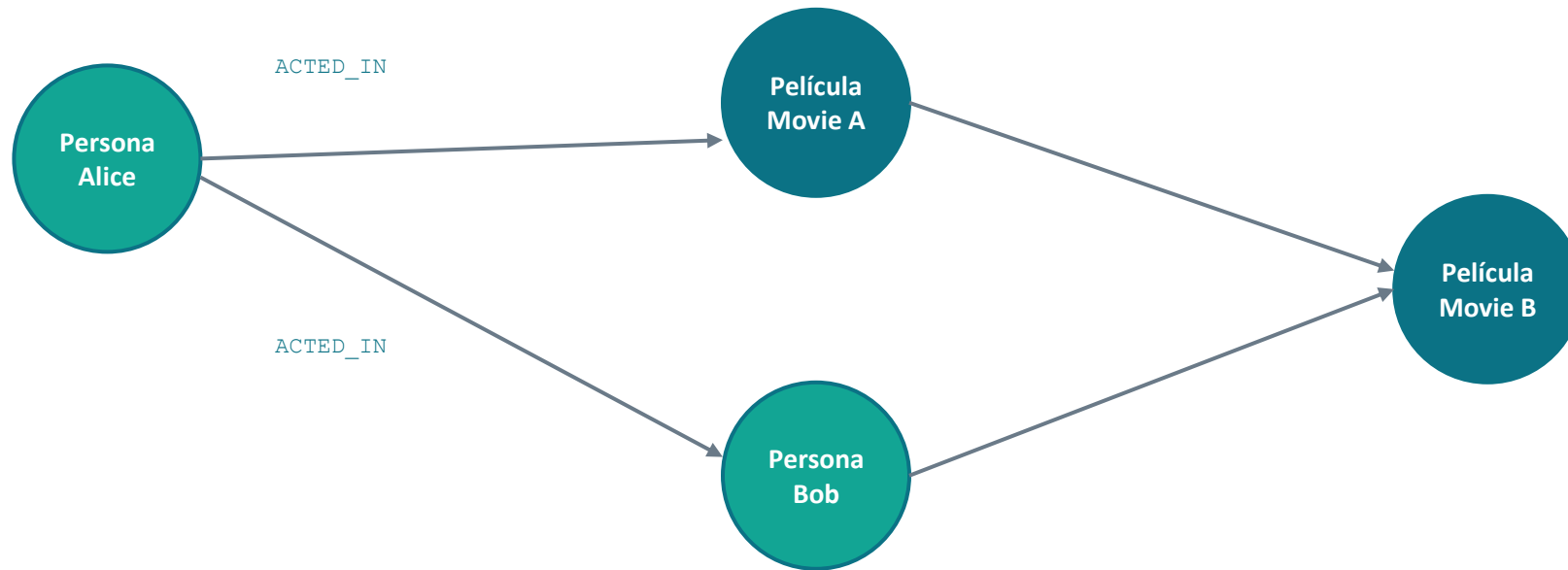
- Servimos el modelo por REST (FastAPI)
- Contratos validados y versionado
- Empaquetado en Docker

Hoy resolvemos

- El cliente necesita consultar metadatos de experimentos y modelos con flexibilidad
- Y navegar relaciones (linaje) entre datos, features y modelos
- REST se queda corto para eso → GraphQL + grafos

¿Qué es un grafo?

Un grafo modela entidades y sus relaciones: nodos (las cosas) conectados por aristas (cómo se relacionan). La relación es tan importante como el dato.



Nodos = entidades · Aristas (dirigidas y con tipo) = relaciones

Definición y tipos

Formalmente, un grafo es una tupla: un conjunto de nodos (vértices) y un conjunto de aristas (arcos) que unen pares de nodos.

No dirigido

La arista no tiene sentido: la relación es mutua ($A-B$).

Dirigido (digrafo)

La arista tiene dirección: nodo padre $\rightarrow$ nodo hijo ($A \rightarrow B$).

Sin peso

Solo importa si existe la conexión.

Con peso

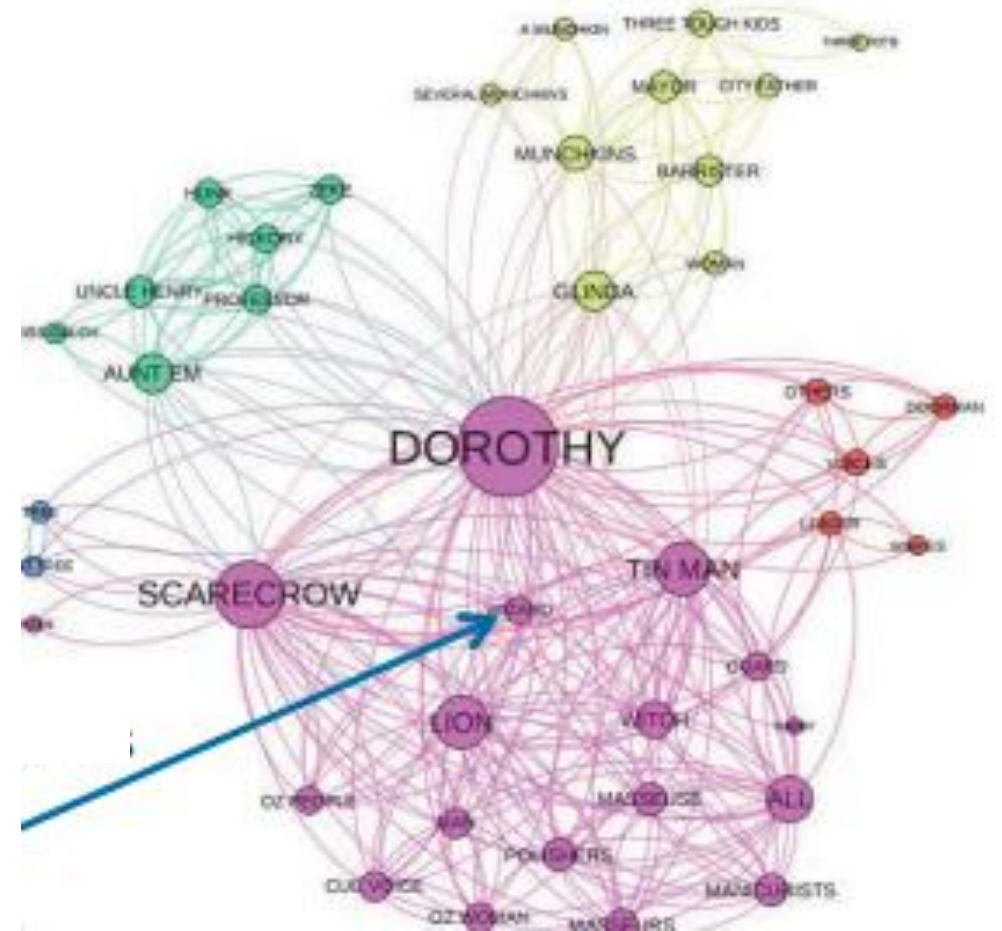
La arista lleva un valor: distancia, tiempo, costo, fuerza.

¿Para qué son útiles los grafos?

Nuestro mundo está explicado por un sinnúmero de redes basadas en relaciones:

- Redes de computadora
- Redes de transporte
- Redes financieras
- Redes de agua o alcantarillado
- Redes políticas
- Redes criminales
- Redes sociales

Análisis de las interacciones de los personajes en las escenas (tamaño) e interacciones naturales entre personajes (color) – Novela Mago de Oz



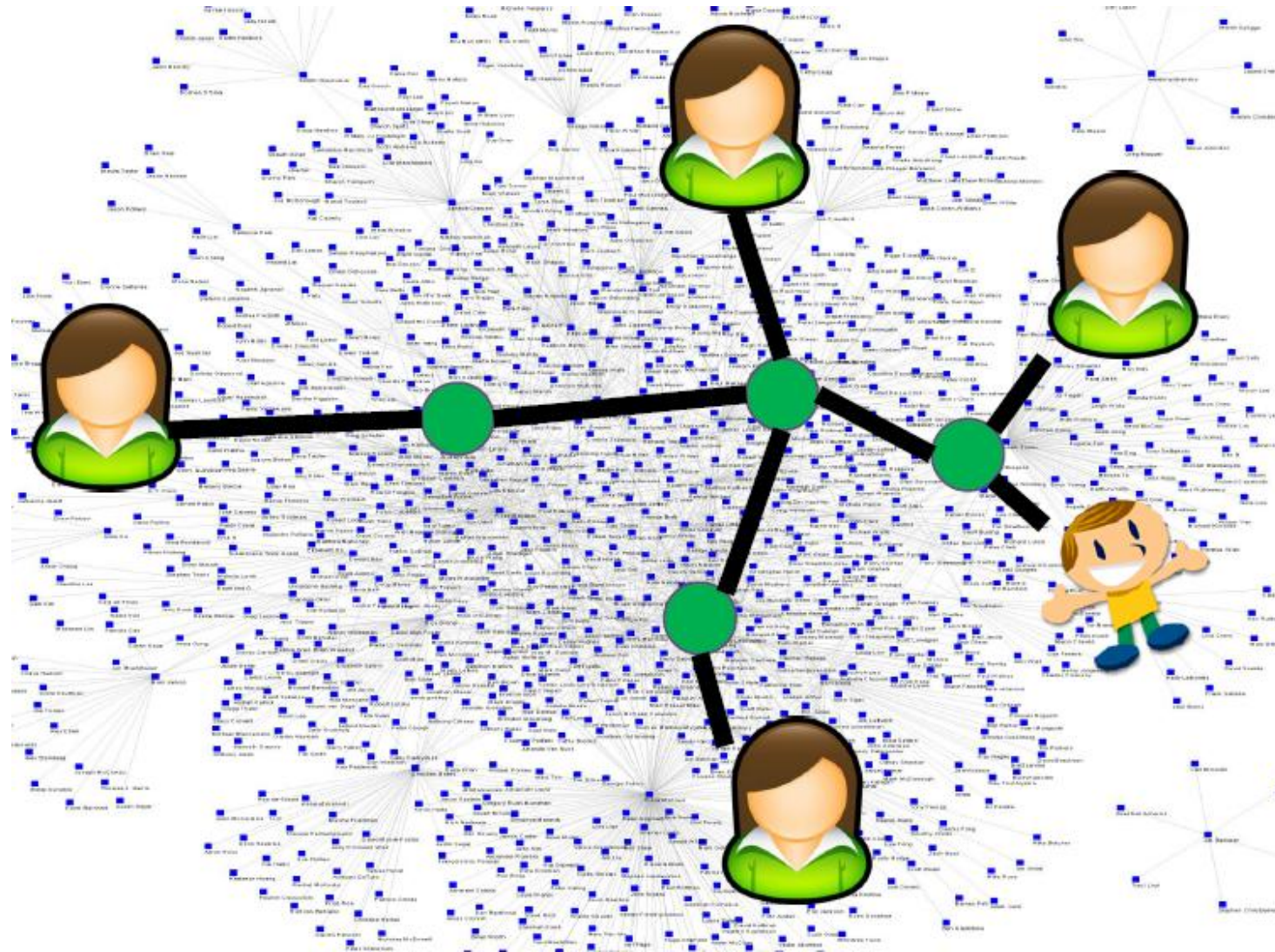
¿Para qué son útiles los grafos?



¿Para qué son útiles los grafos?



¿Para qué son útiles los grafos?



¿Por qué importan en los datos?



La relación es el dato

En muchos problemas, lo valioso son las conexiones, no las filas: quién se conecta con quién.



Recorridos multi-hop

Preguntas de varios saltos ('amigos de amigos', caminos) son naturales y rápidas en un grafo.



Casos reales

Redes sociales, recomendación, detección de fraude, grafos de conocimiento... y linaje de datos/modelos.



Linaje y trazabilidad

El recorrido dato → feature → modelo → despliegue es, literalmente, un grafo.

Además de guardar relaciones, permiten inferir



¿Existe un camino?

Saber si A se conecta con B (directa o indirectamente).



Camino más corto

El recorrido de menor costo entre A y B (shortest path).



Particionar

Dividir el grafo en subconjuntos conectados (comunidades).



Flujo máximo / corte mínimo

La forma más eficiente de dividir o transportar por la red.

Ejemplo clásico: modelar el transporte como digrafo con pesos (tiempo/distancia) y resolver el camino más corto casa → universidad.

Cómo se representa: matriz vs lista de adyacencia

Matriz de adyacencia

Tabla $V \times V$: $\text{celda}[i,j]=1$ si hay arco $i \rightarrow j$.

- Consultar un arco: $O(1)$ — muy rápido
- Memoria: $O(V^2)$ — crece al cuadrado
- Insertar/borrar un vértice: $O(V^2)$

Facebook Colombia como matriz ≈ 1387 TB

Lista de adyacencia

Cada nodo guarda la lista de sus vecinos.

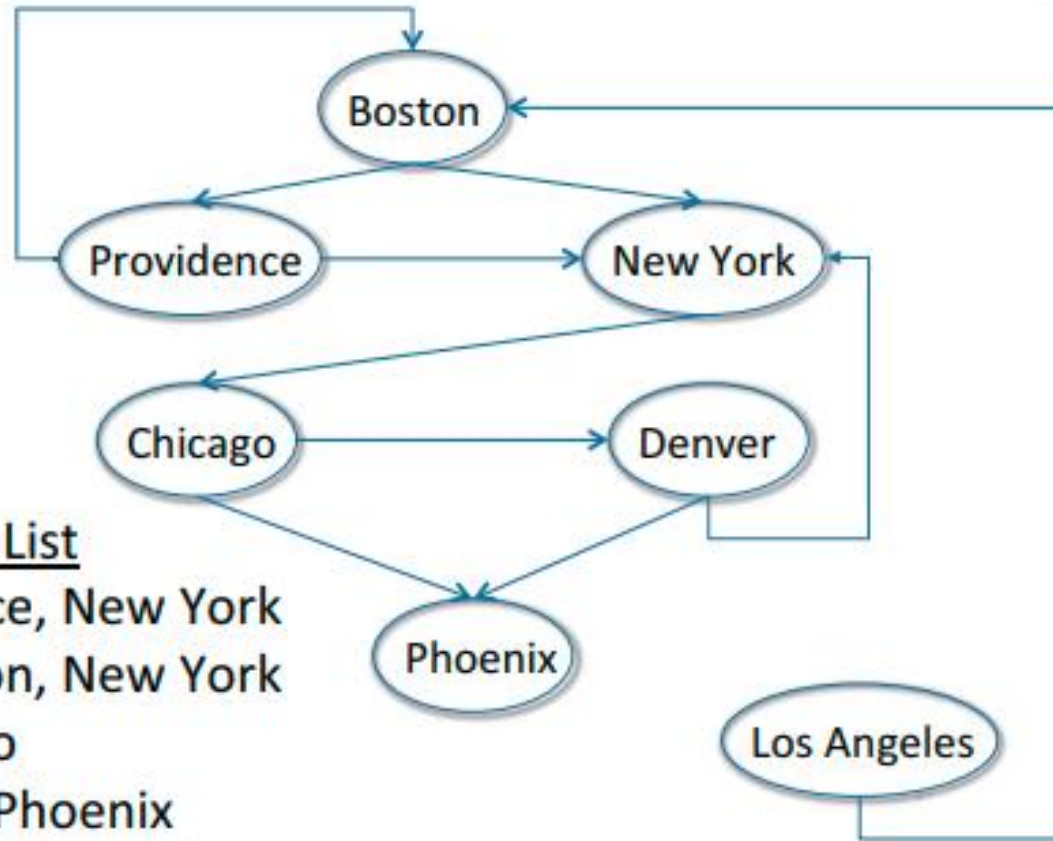
- Consultar un arco: $O(n)$ sobre los vecinos
- Memoria: proporcional a nodos + aristas
- Ideal para grafos dispersos (pocos arcos)

El mismo Facebook Colombia como lista ≈ 300 MB (entra en RAM)

Implementación de grafos

- Para implementar y usar grafos necesitaríamos:
 - Nodos
 - Arcos
 - Juntarlos para crear caminos
- Una vez creados los caminos podemos:
 - Buscar el camino más corto
 - Buscar el camino más óptimo

Ejemplo



Adjacency List

Boston: Providence, New York

Providence: Boston, New York

New York: Chicago

Chicago: Denver, Phoenix

Denver: Phoenix, New York

Los Angeles: Boston

Phoenix:

Recorridos para el camino más corto: DFS vs BFS



DFS — en profundidad

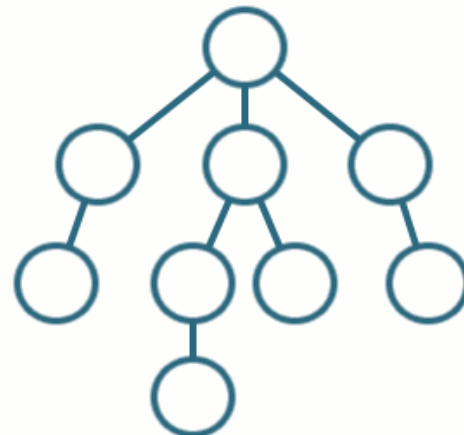
- Sigue un camino hasta el final antes de retroceder
- Si se queda sin opciones, vuelve al nodo previo
- Usa una pila (o recursión)
- Bueno para '¿existe un camino?'



BFS — en anchura

- Explora nivel por nivel desde el origen
- Encuentra el camino más corto en nº de saltos
- Usa una cola
- Base de muchos algoritmos de grafos

DFS

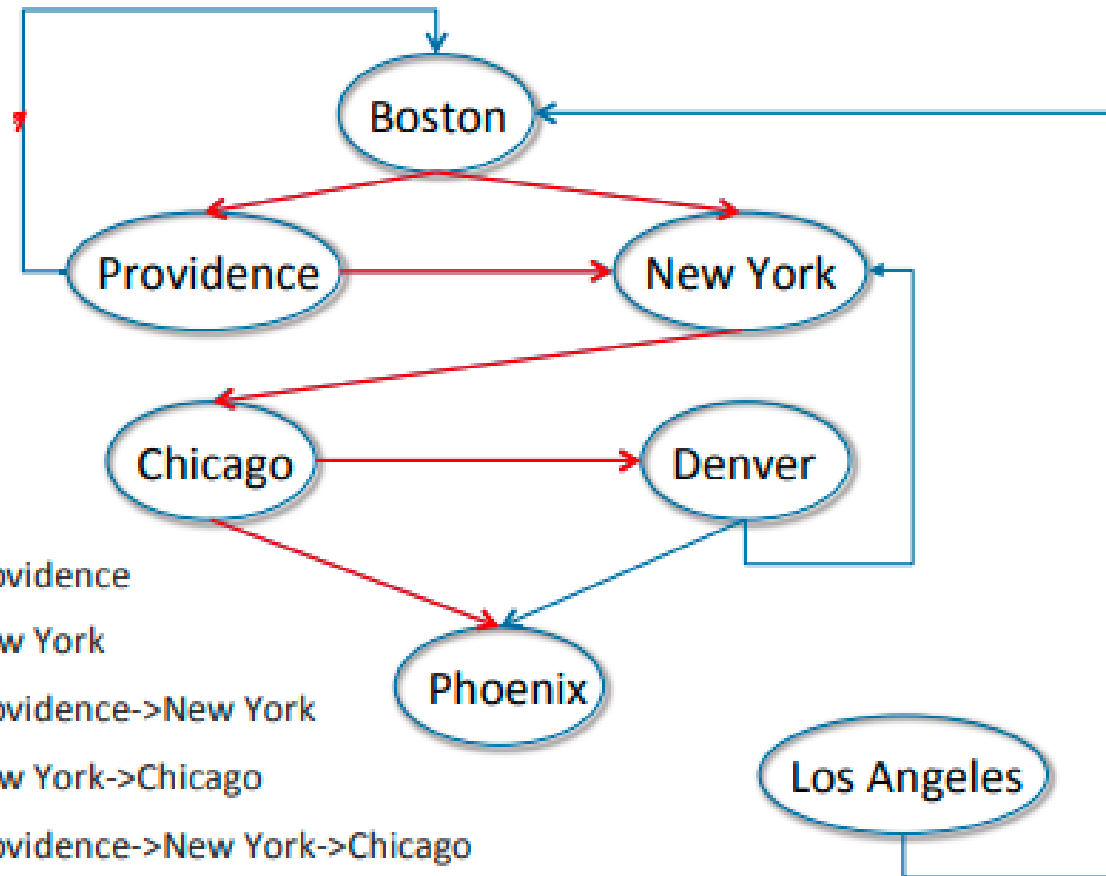


BFS



Ejemplo – Breadth-First Search – Boston a Phoenix

<https://colab.research.google.com/drive/1dywndmekYWE80AJoOLgTk8F1aGj09Gx?usp=sharing>



Current BFS path: Boston

Current BFS path: Boston->Providence

Current BFS path: Boston->New York

Current BFS path: Boston->Providence->New York

Current BFS path: Boston->New York->Chicago

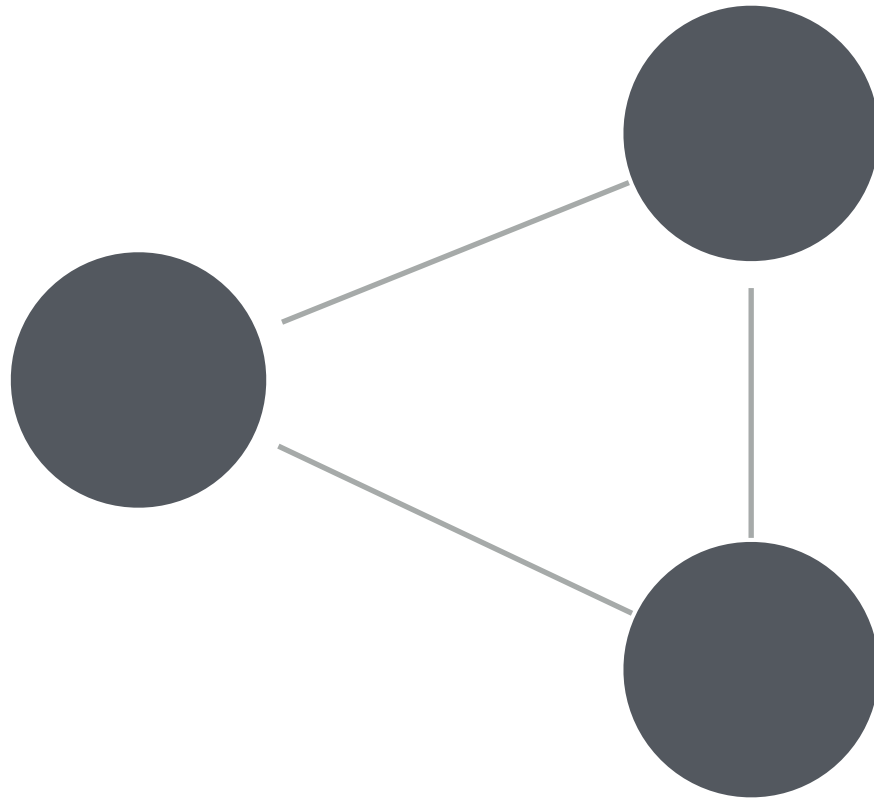
Current BFS path: Boston->Providence->New York->Chicago

Current BFS path: Boston->New York->Chicago->Denver

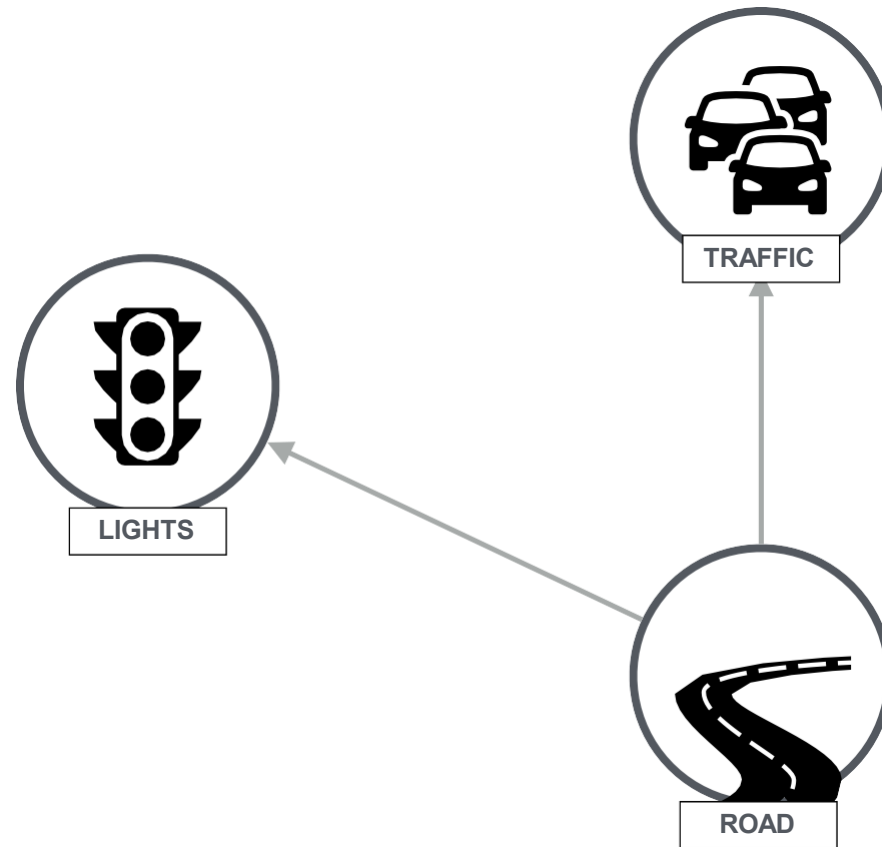
Current BFS path: Boston->New York->Chicago->Phoenix

Shortest path from Boston to Phoenix is Boston->New York->Chicago->Phoenix

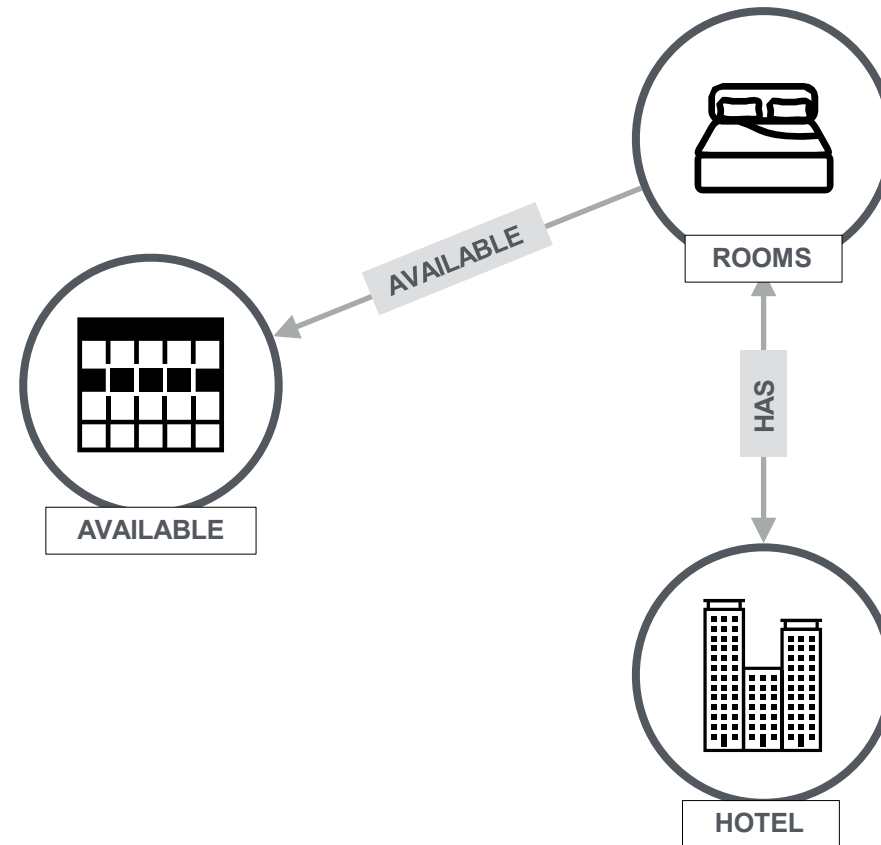
Grafos para almacenar datos - datos conectados

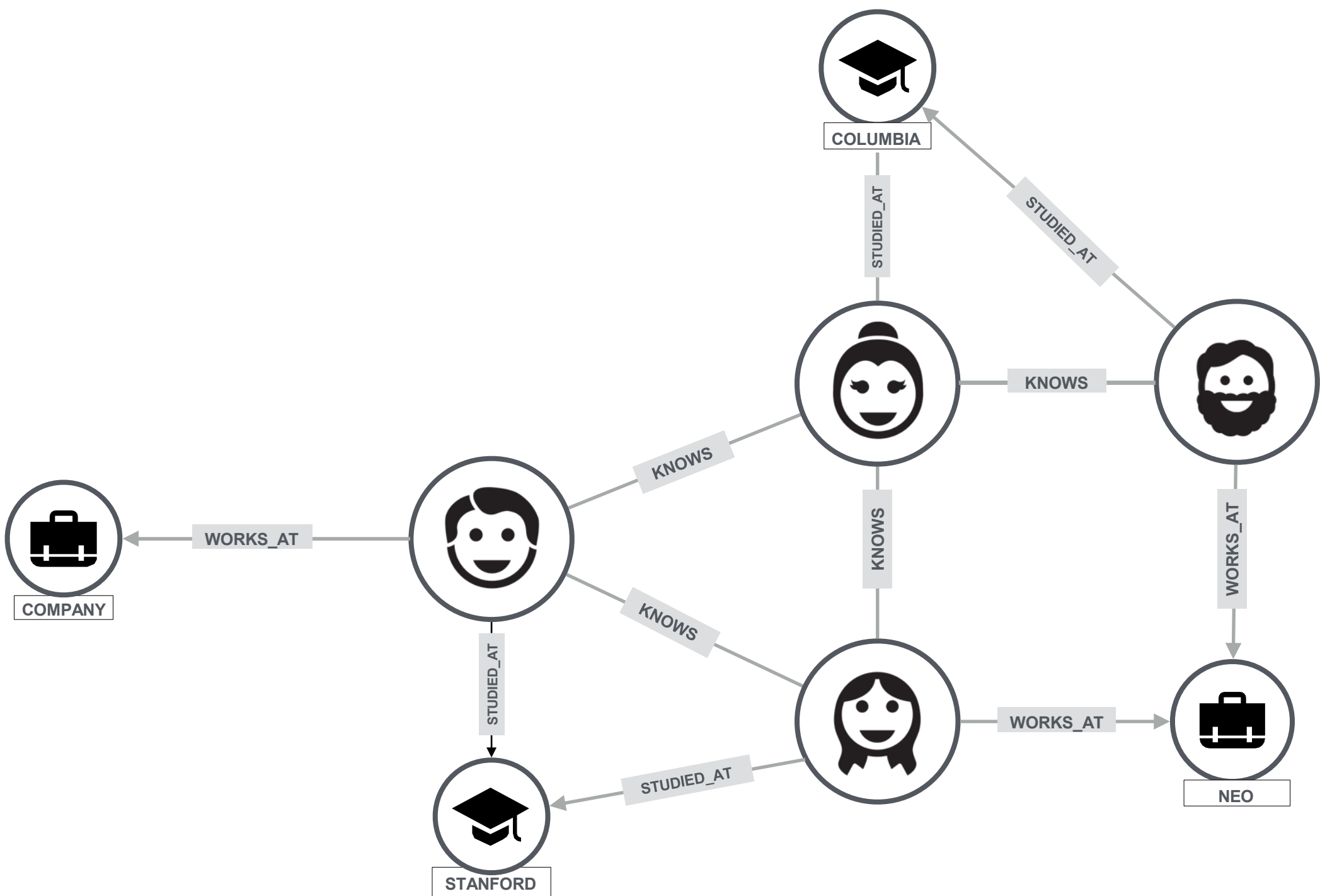


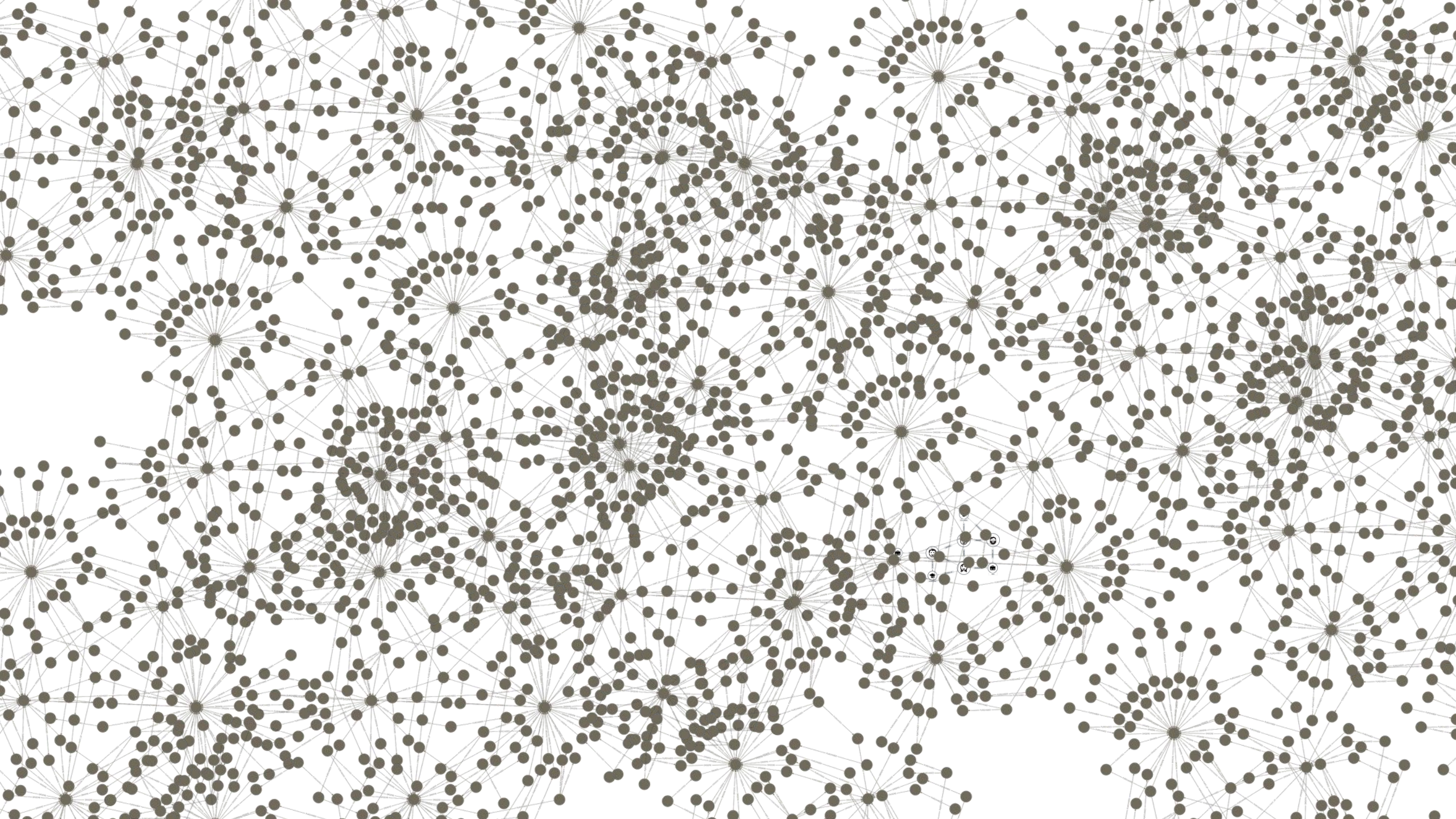
Grafos para almacenar datos - datos conectados



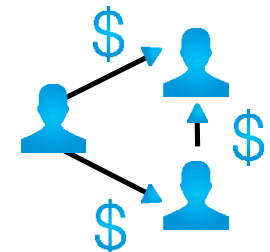
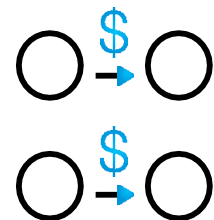
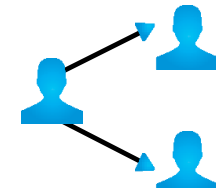
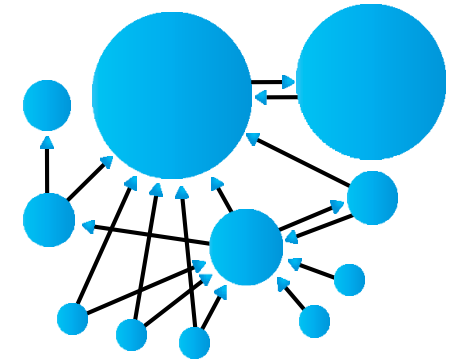
Grafos para almacenar datos - datos conectados



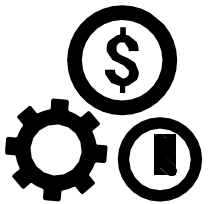




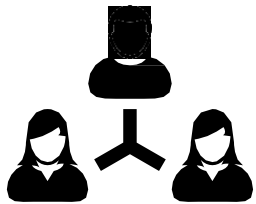
Empresas con grandes volúmenes de datos – en grafos



Practicamente en todas las areas tenemos grafos



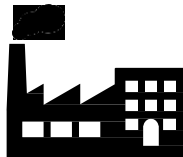
Finance



Social networks



**HR &
Recruiting**



**Manufacturing
& Logistics**



Health Care

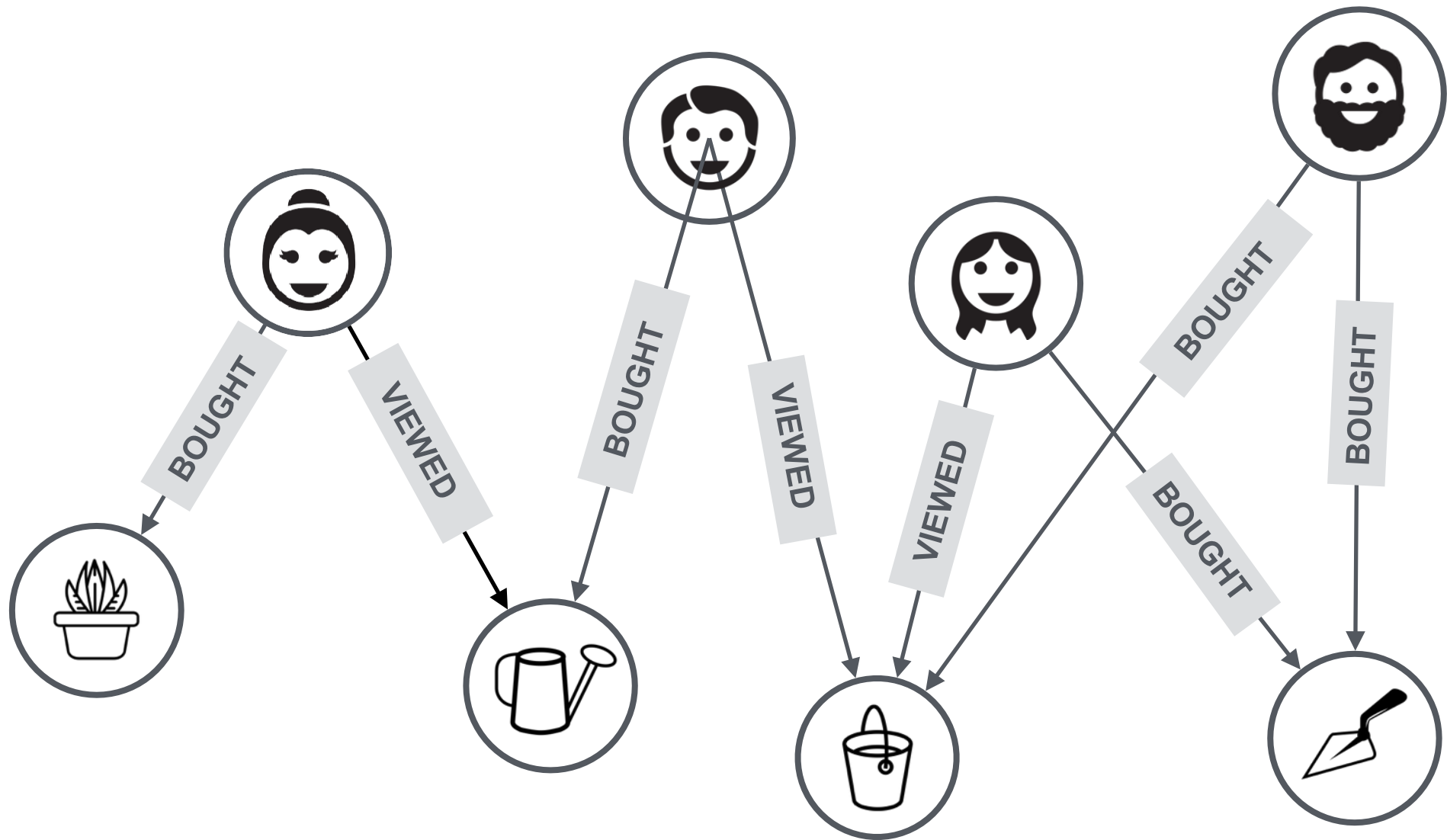


Telco

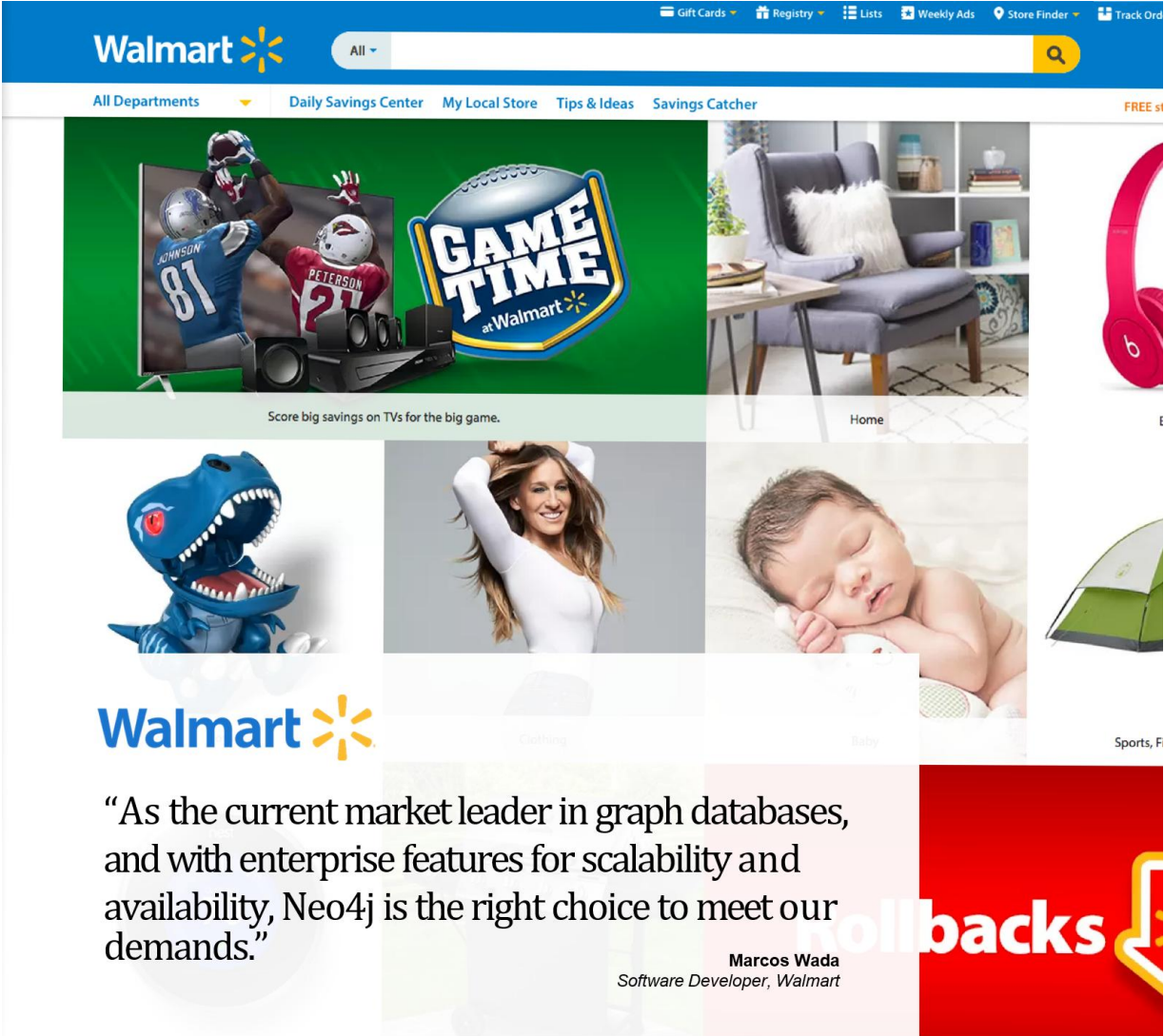


Retail

User cases – Recomendaciones en tiempo real



User cases – Recomendaciones en tiempo real



The screenshot displays the Walmart website's homepage. At the top, the Walmart logo is prominent, followed by navigation links for Gift Cards, Registry, Lists, Weekly Ads, Store Finder, and Track Order. Below this, a secondary navigation bar includes All Departments, Daily Savings Center, My Local Store, Tips & Ideas, and Savings Catcher. The main content area features several promotional banners and product images. A large banner at the top left promotes 'GAME TIME at Walmart' with images of football players. To the right, there's a section for 'Home' featuring a blue armchair. Below the game time banner, there's a section for 'Score big savings on TVs for the big game.' Other visible sections include a blue dinosaur toy, a woman in a white top, a baby sleeping, and a green tent. The bottom of the page shows the Walmart logo again, followed by a quote from Marcos Wada, Software Developer at Walmart, and a red banner with the word 'Rollbacks'.

Walmart

All Departments Daily Savings Center My Local Store Tips & Ideas Savings Catcher

Score big savings on TVs for the big game.

Home

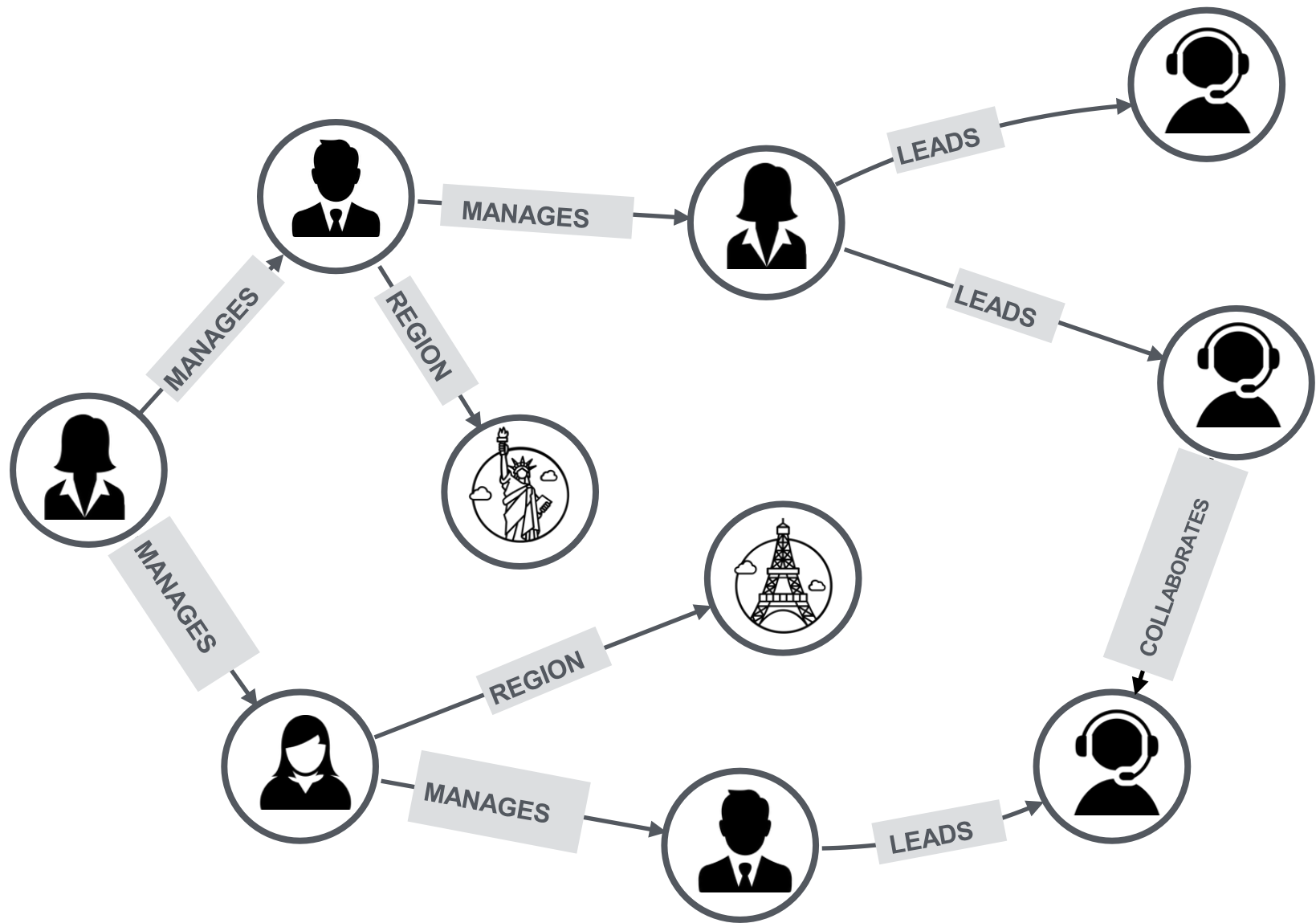
Walmart

Rollbacks

“As the current market leader in graph databases, and with enterprise features for scalability and availability, Neo4j is the right choice to meet our demands.”

Marcos Wada
Software Developer, Walmart

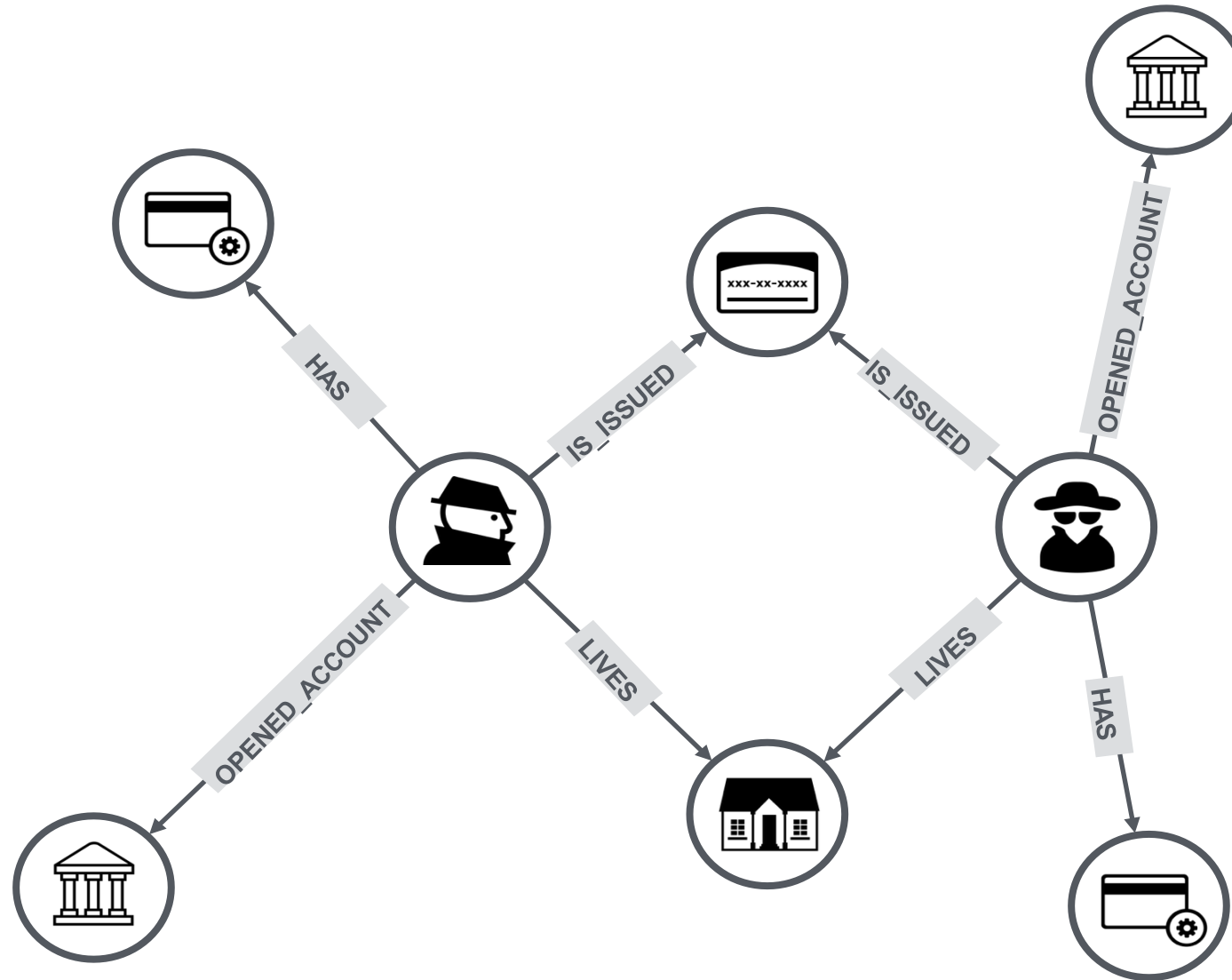
User cases – Gestión de datos maestros



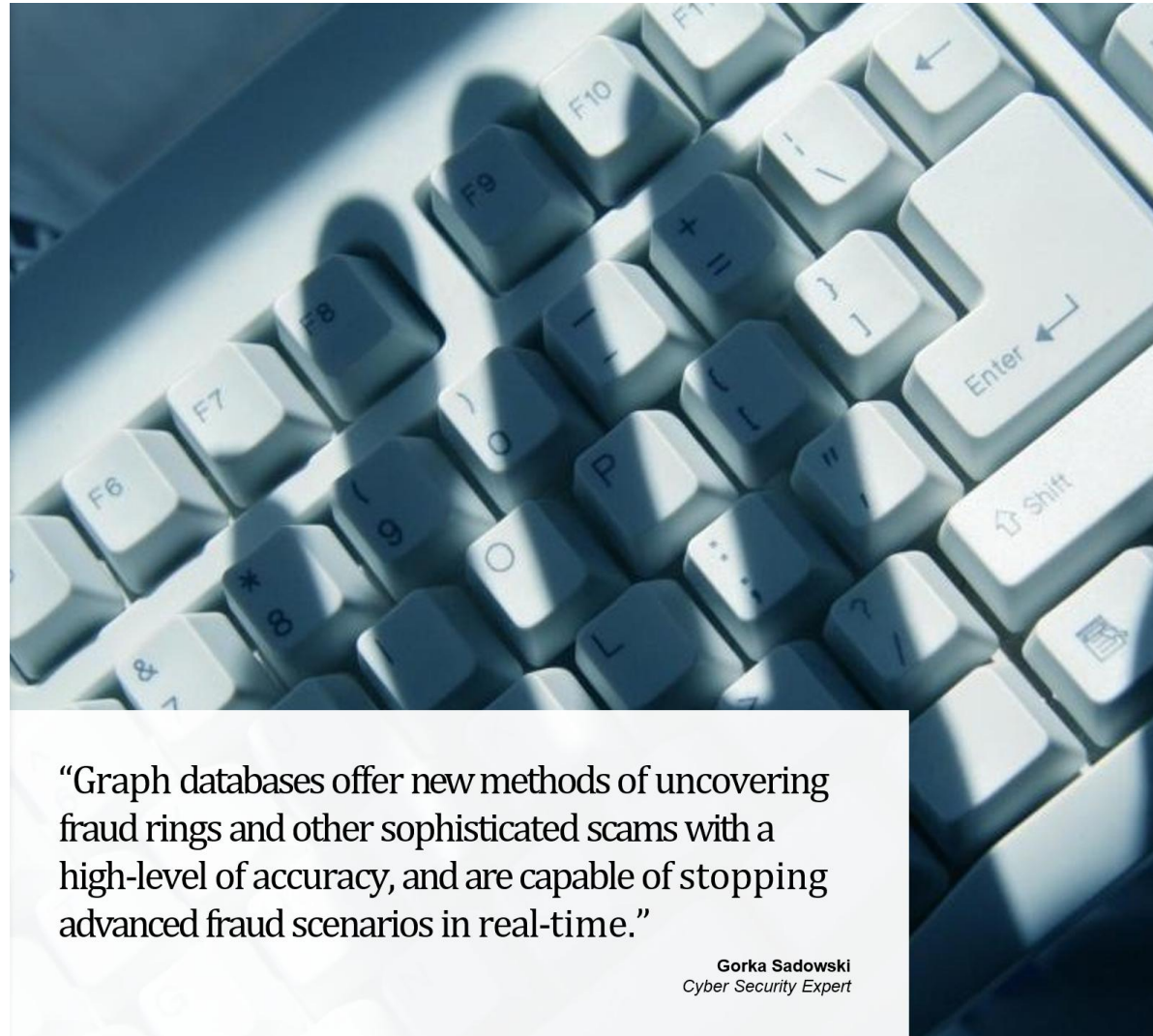
User cases – Gestión de datos maestros



User cases – Detección de fraudes



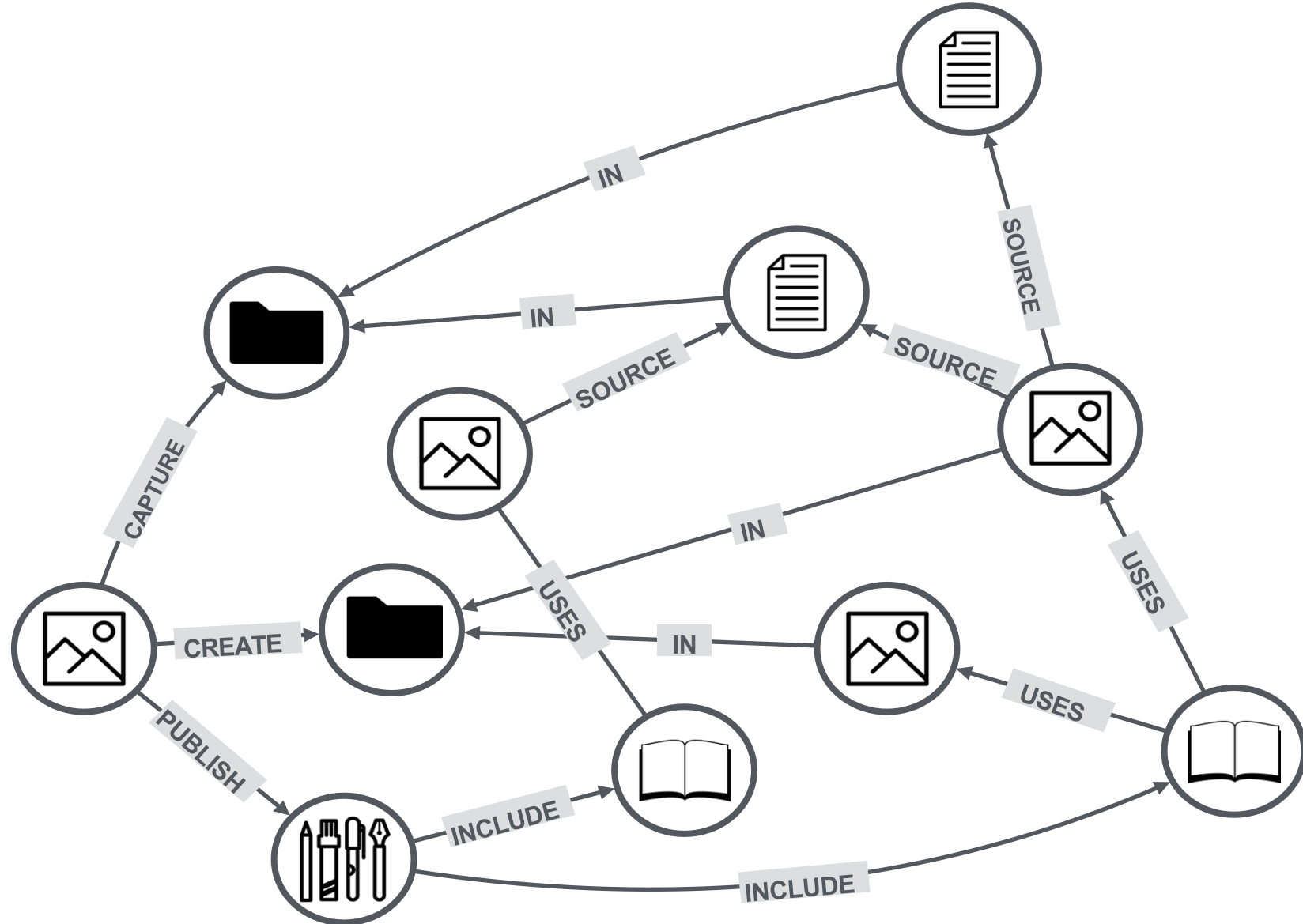
User cases – Detección de fraudes



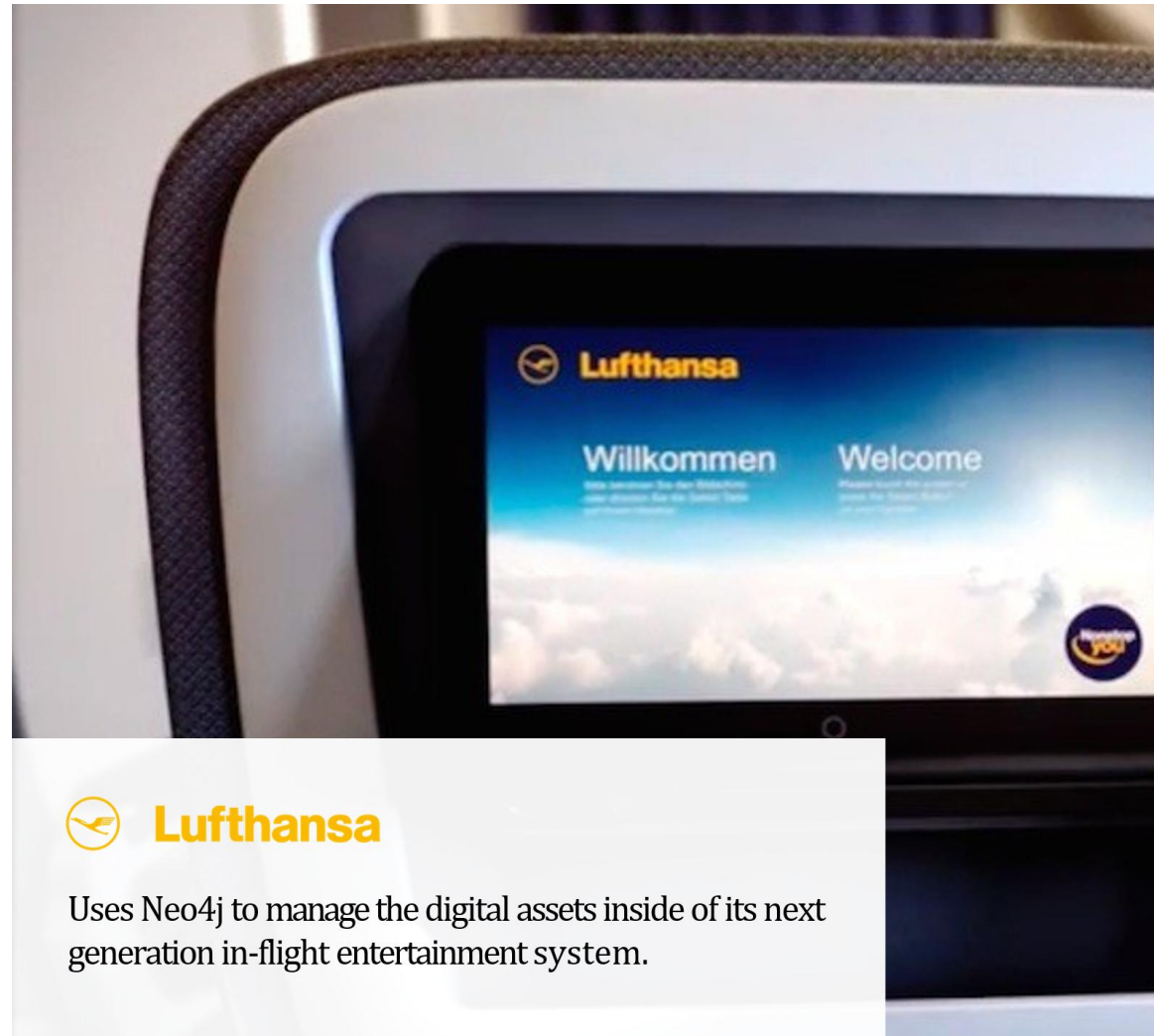
“Graph databases offer new methods of uncovering fraud rings and other sophisticated scams with a high-level of accuracy, and are capable of stopping advanced fraud scenarios in real-time.”

Gorka Sadowski
Cyber Security Expert

User cases – Búsqueda basada en grafos

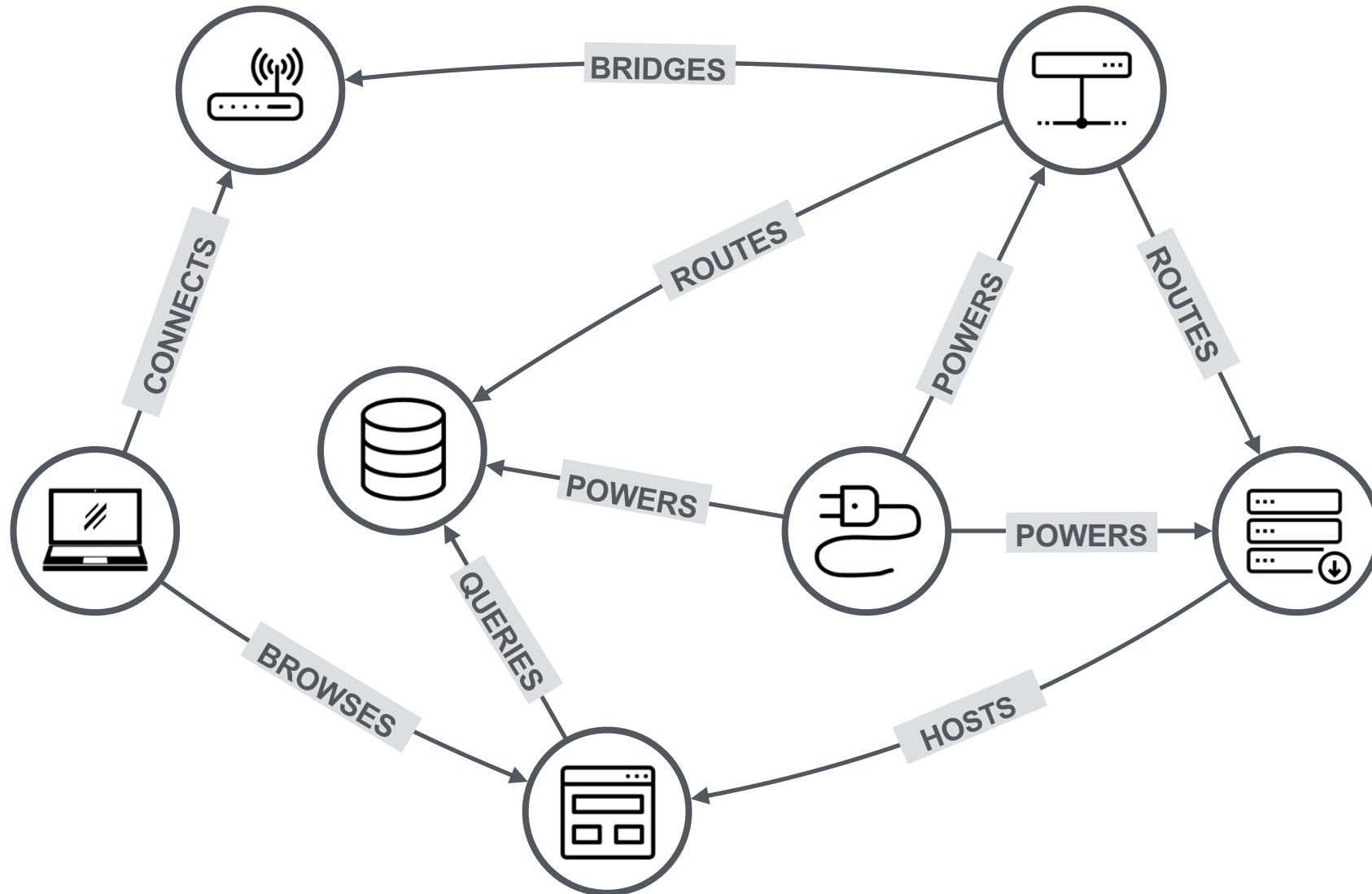


User cases – Búsqueda basada en grafos



Uses Neo4j to manage the digital assets inside of its next generation in-flight entertainment system.

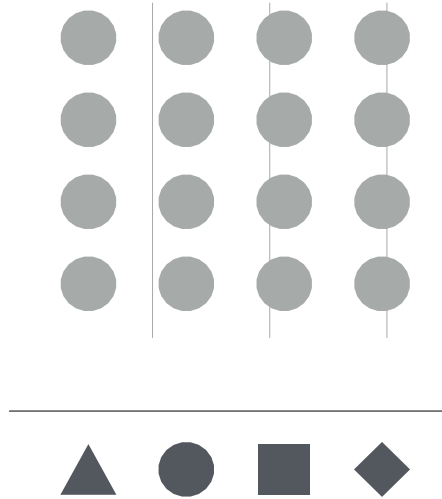
User cases – Red y operaciones TI



User cases – Red y operaciones TI



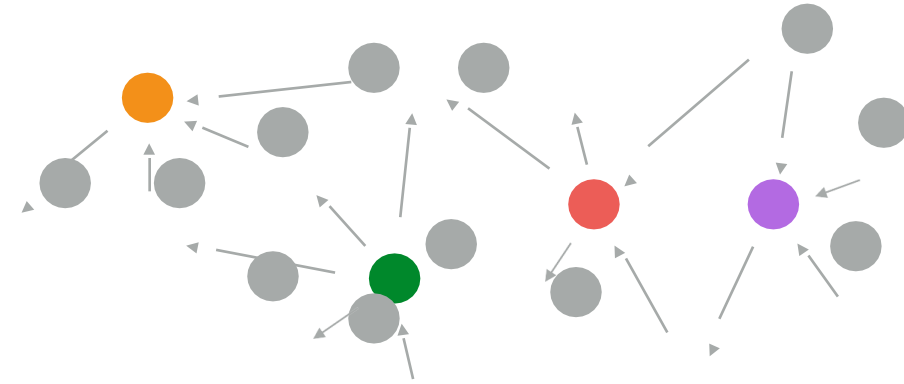
Con grafos tenemos una nueva forma de representar datos



Relational
Database



Graph
Database



Bueno para:

- Estructuras de datos bien entendidas que no cambian con demasiada frecuencia
- Problemas conocidos que involucran partes discretas de los datos, o conectividad mínima

- Sistemas dinámicos: donde la topología de los datos es difícil de predecir
- Requisitos dinámicos: la evolución con el negocio
- Problemas en los que las relaciones en los datos aportan significado y valor

El mismo dominio: tablas vs grafo

Relacional (tablas)

Las relaciones se guardan de forma indirecta: claves foráneas y tablas de unión. Para 'navegar' de una entidad a otra haces un JOIN.

```
Persons(id, name)
Movies(id, title)
Relationships(
    person_id, movie_id, type)
-- navegar = JOINS
```

Grafo (Neo4j)

Las relaciones son entidades de primera clase, con tipo y dirección. El modelo se parece al dominio y navegar es seguir punteros.

```
(:Person {name})
  -[:ACTED_IN]->
(:Movie {title})

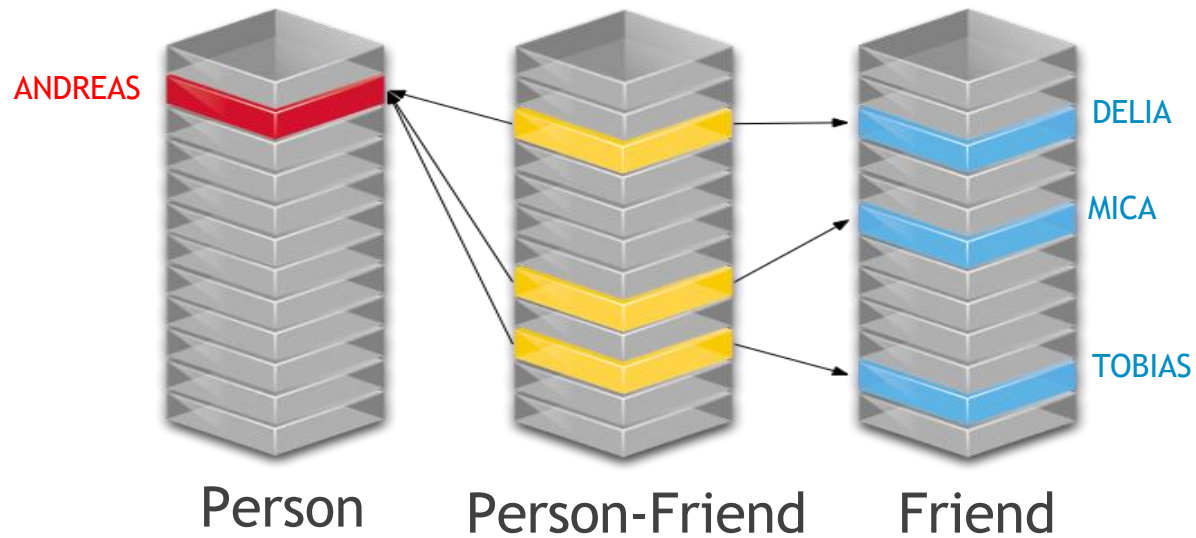
// navegar = seguir la arista
```

¿Por qué usar bases de datos en grafos?

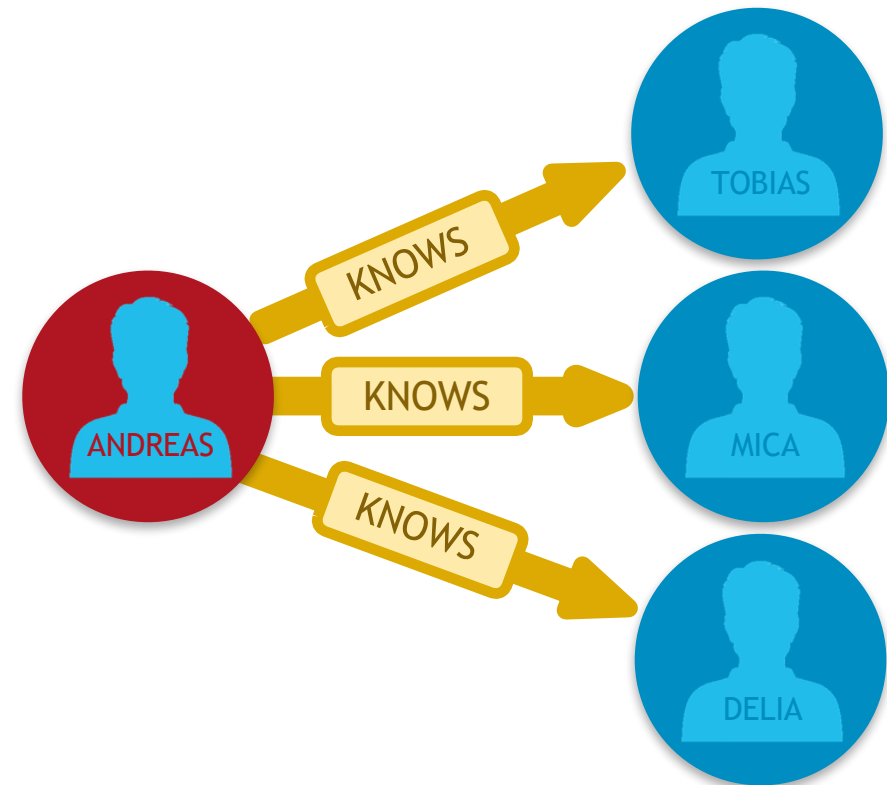
- Principalmente tendremos tres grandes ventajas:
 - Representaciones más intuitivas
 - Procesamiento más rápido
 - Versatilidad y agilidad en las consultas

Procesamiento más rápido

Relational Model



Graph Model



Index free adjacency

Procesamiento más rápido



“We found Neo4j to be literally **thousands of times faster** than our prior MySQL solution, with queries that require 10-100 times less code. Today, Neo4j provides eBay with functionality that was previously impossible.”

- Volker Pacher, Senior Developer

“Minutes to milliseconds” performance
Queries up to 1000x faster than RDBMS or other NoSQL



Versatilidad y agilidad en las consultas

Un modelo naturalmente adaptativo

+

Un lenguaje de consulta diseñado
para la Conectividad

Versatilidad y agilidad en las consultas

Un modelo naturalmente adaptativo

+

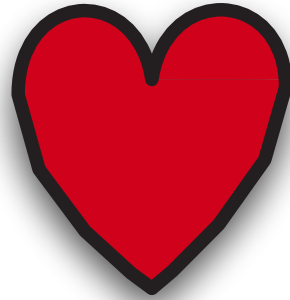
Un lenguaje de consulta diseñado
para la Conectividad

Veamos un ejemplo:

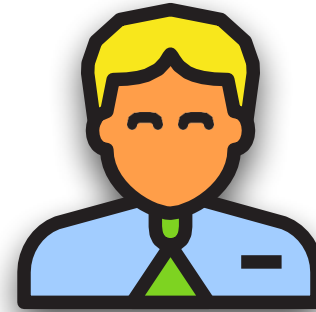
Ann Loves Dan



Ann

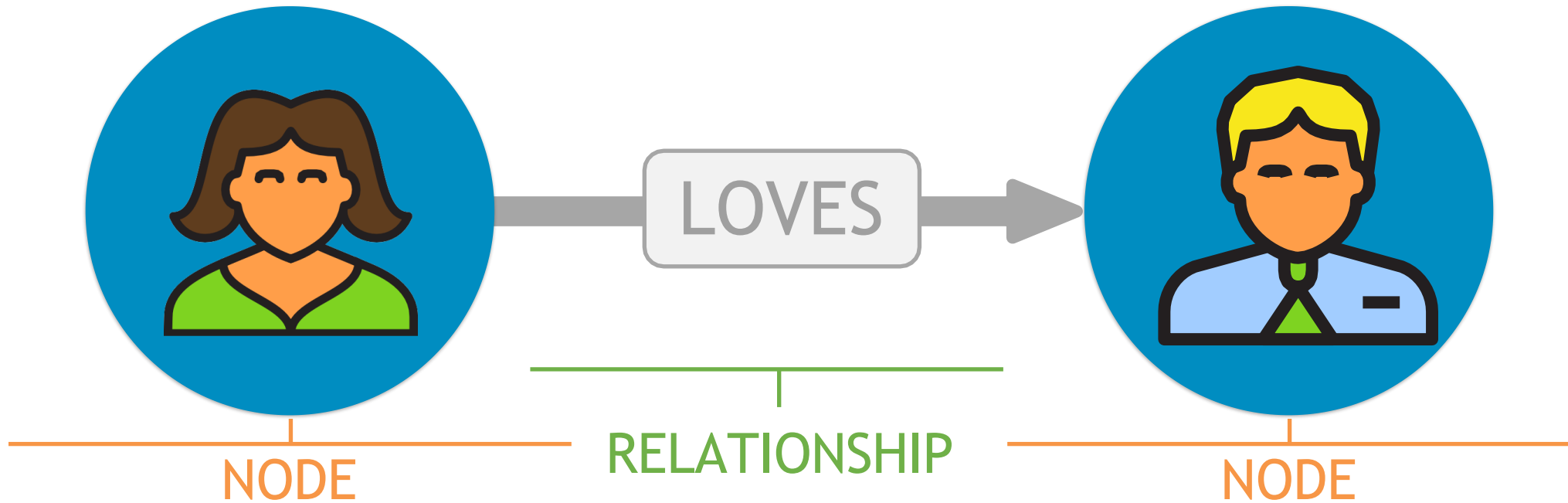


Loves



Dan

Ann Loves Dan



Las relaciones son direccionales

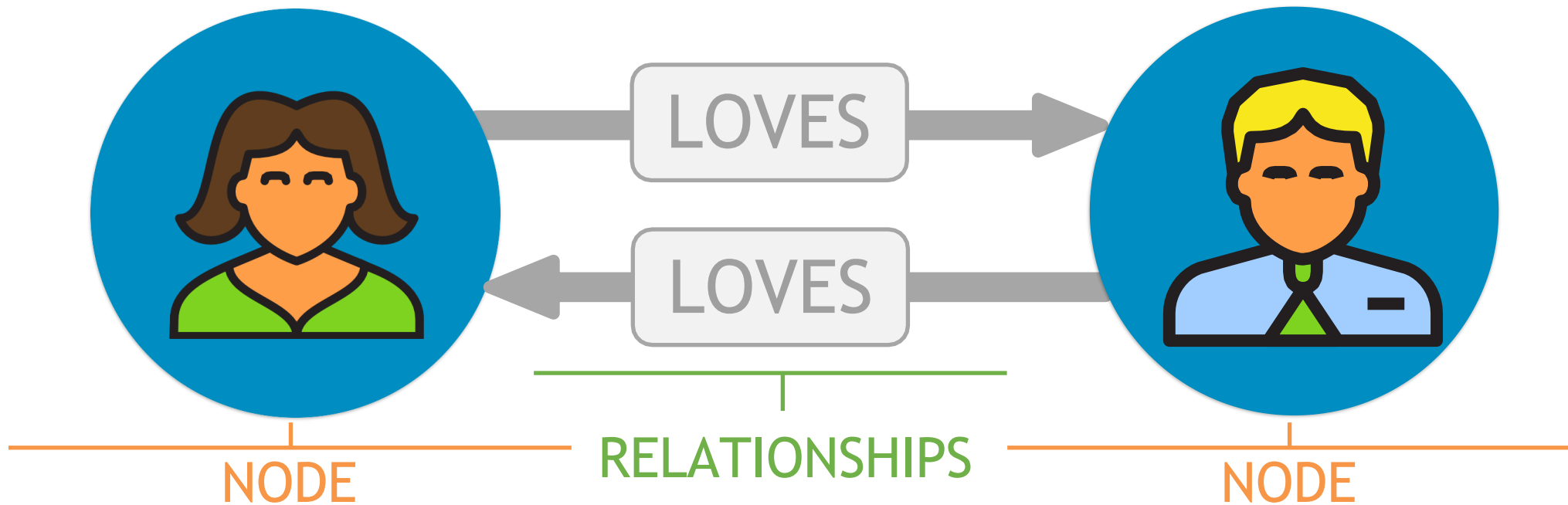
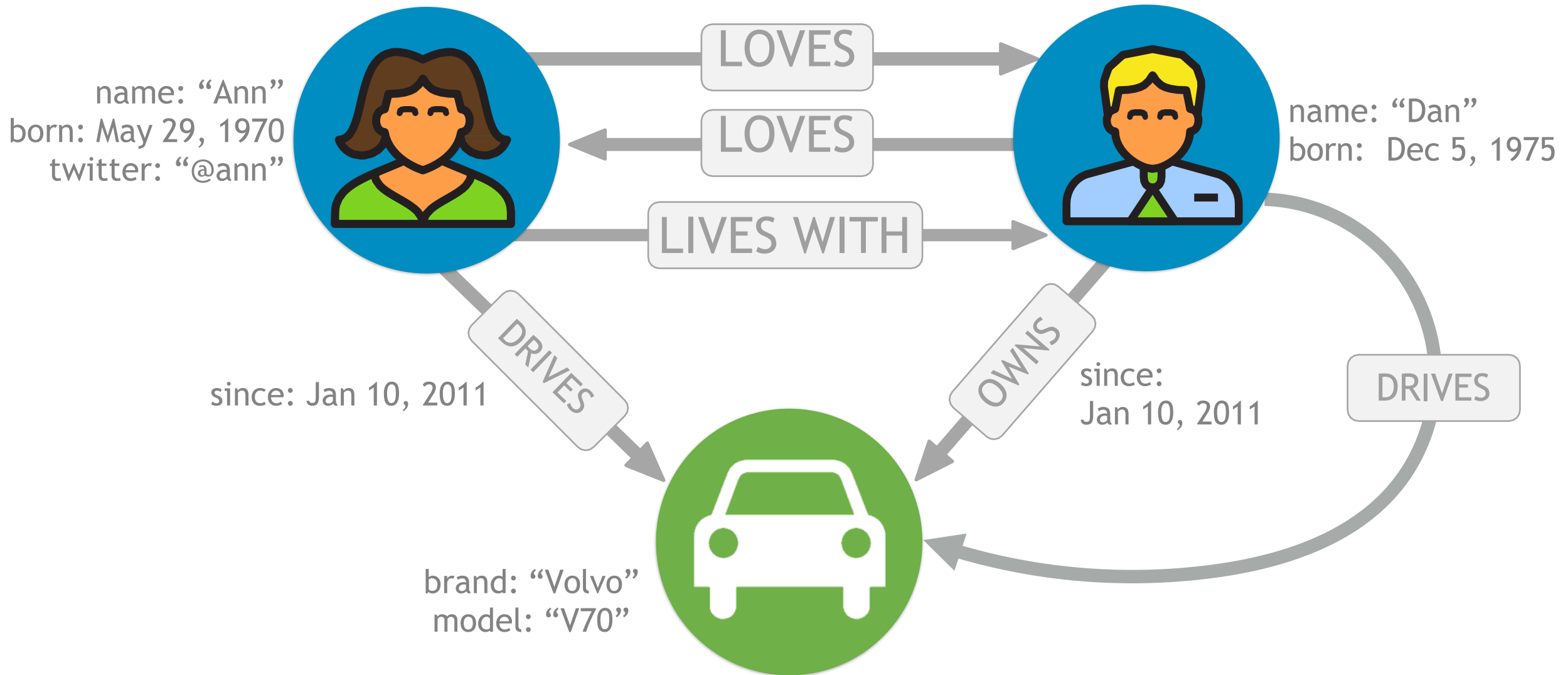
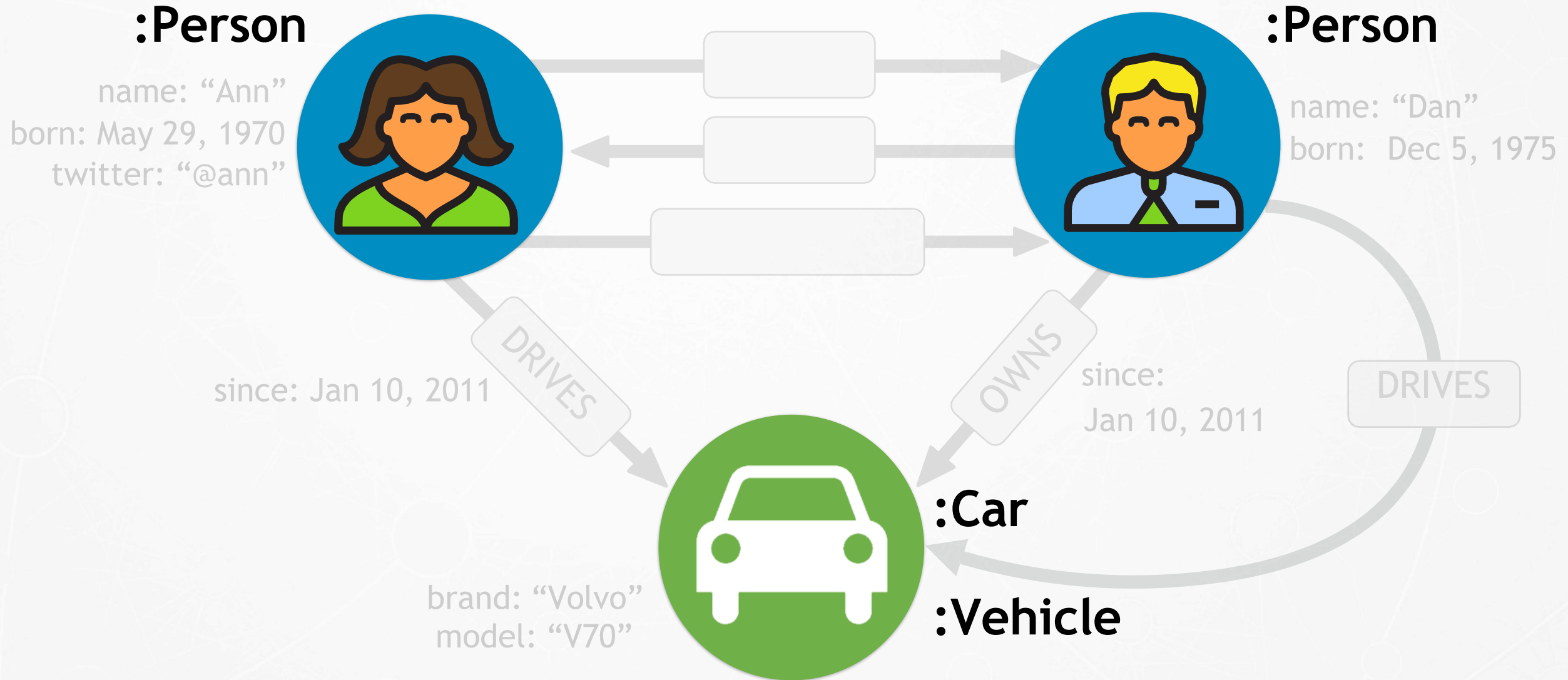


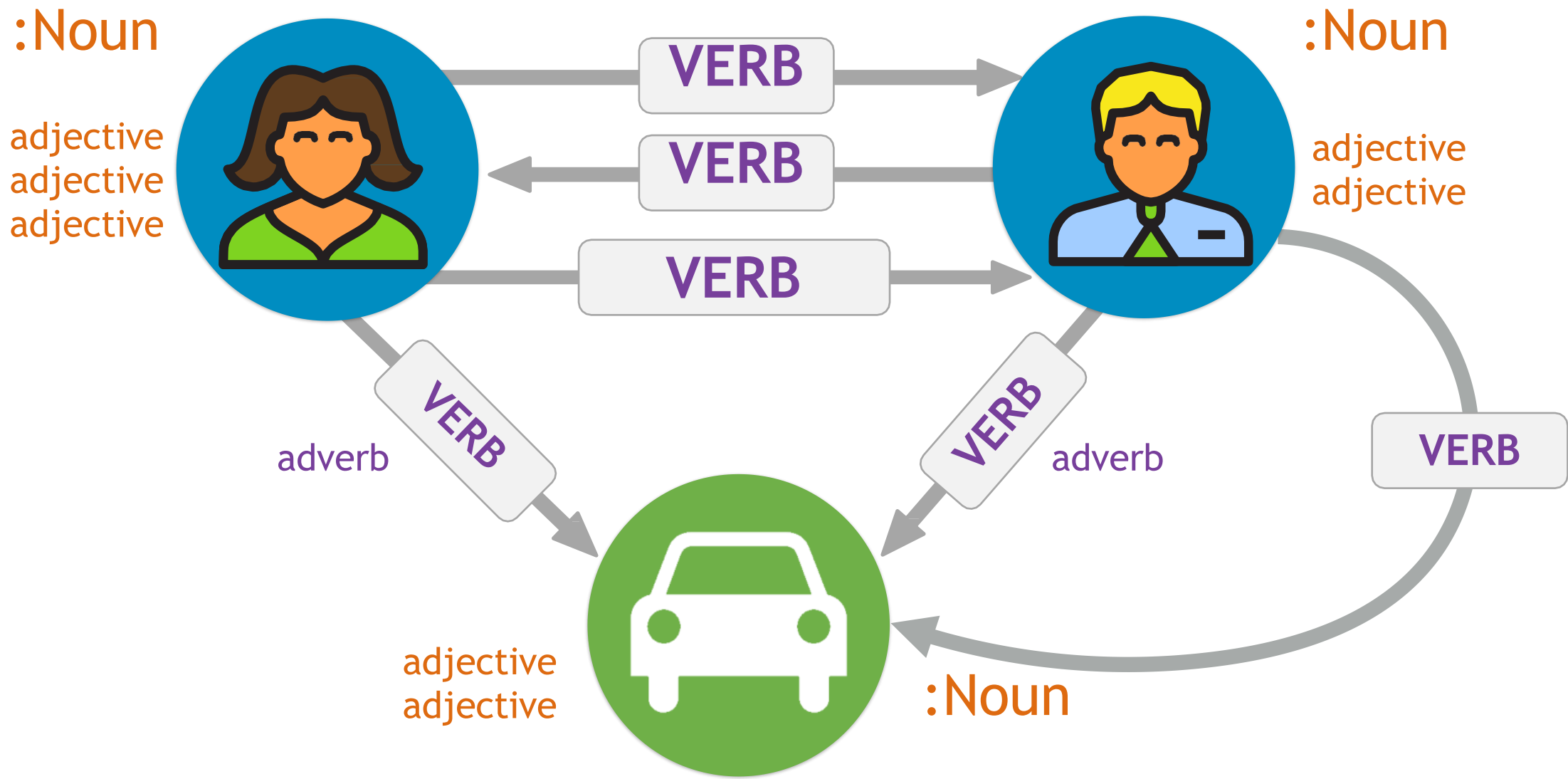
Gráfico de propiedades detallado



Grafo de propiedades etiquetado



Busca semejanzas a lenguajes naturales



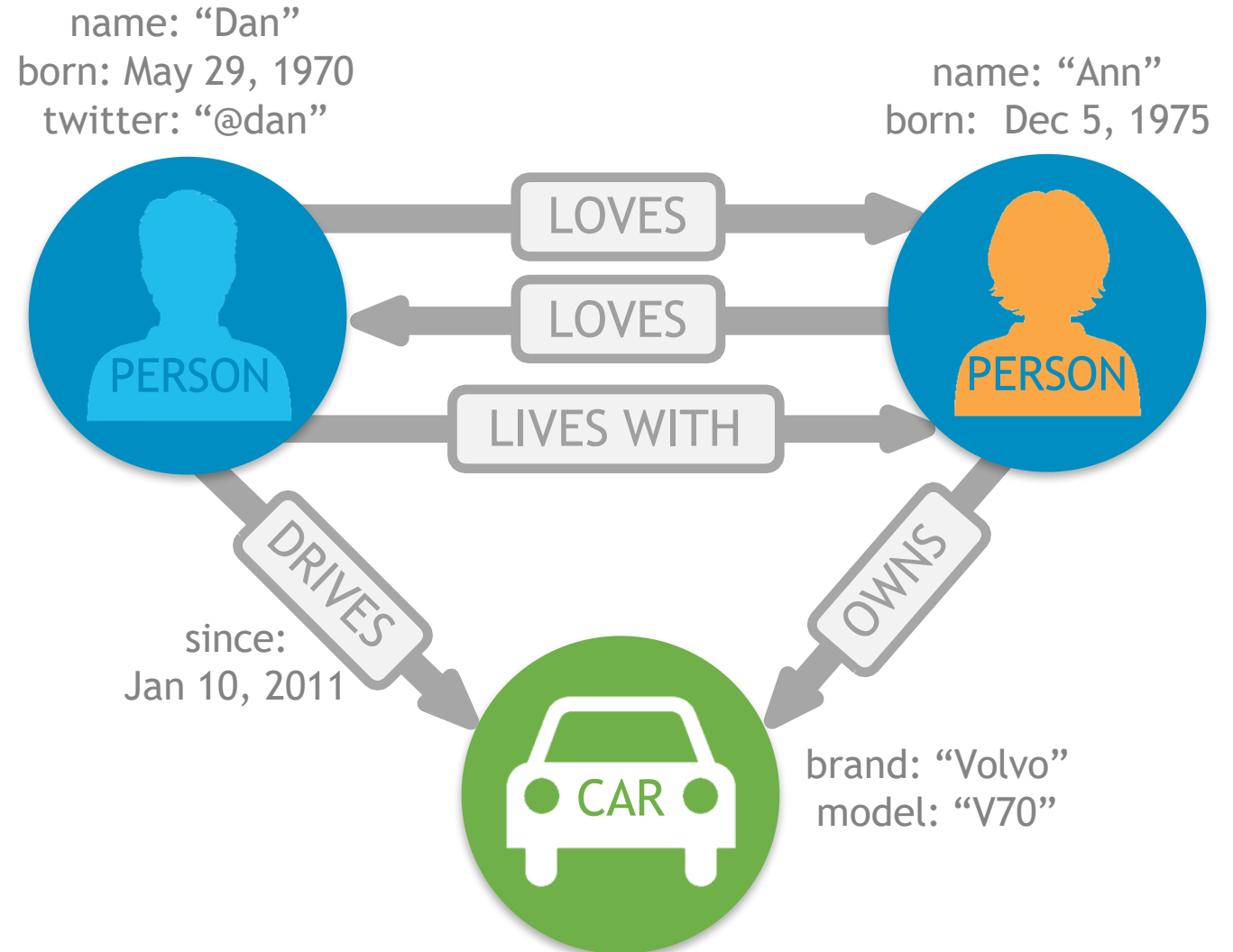
Componentes del modelo de grafo

Nodos

- Los objetos en el grafo
- Puede tener propiedades de valor nominal
- Puede etiquetarse

Relaciones

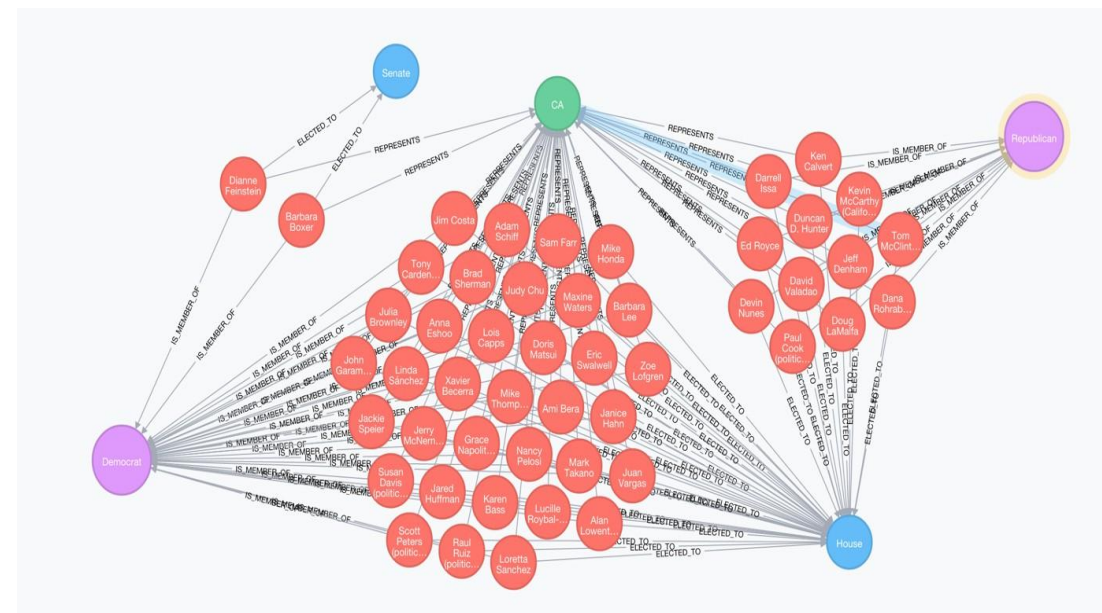
- Relacionar nodos por tipo y dirección
- Puede tener propiedades de valor nominal



Usando neo4j – data base usando grafos



- Modelo de datos de grafos de propiedades
- Nodos y relaciones
- Procesamiento nativo de grafos
- Lenguaje de consultas propio (Cypher)



Neo4j – Características clave del producto



Almacenamiento nativo de grafos

Garantiza la consistencia y el rendimiento de los datos

Procesamiento nativo de grafos

Millones de saltos por segundo, en tiempo real

Modelado de datos "compatible con pizarras"

Modelar los datos tal como ocurren de forma natural

Alta integridad de datos

Transacciones totalmente ACID

Lenguaje de consultas potente y expresivo

Requiere de 10 a 100 veces menos código que SQL

Escalabilidad y alta disponibilidad

Escalado vertical y horizontal optimizado para grafos

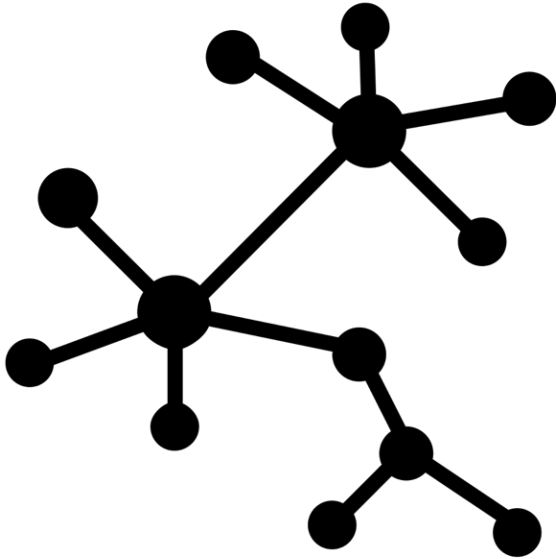
ETL incorporado

Importación fluida desde otras bases de datos

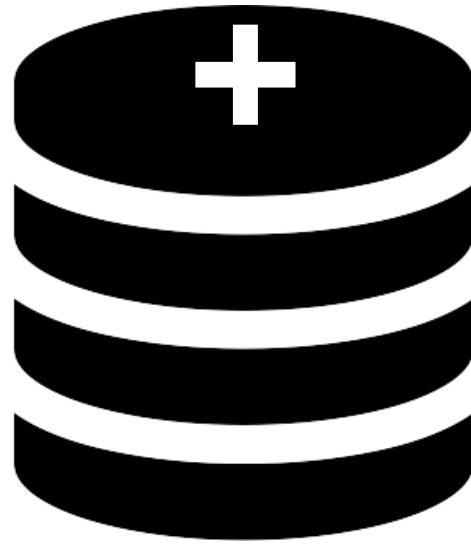
Integración

Controladores y APIs para lenguajes populares

¿Cómo se usa Neo4j?



CREAR MODELO

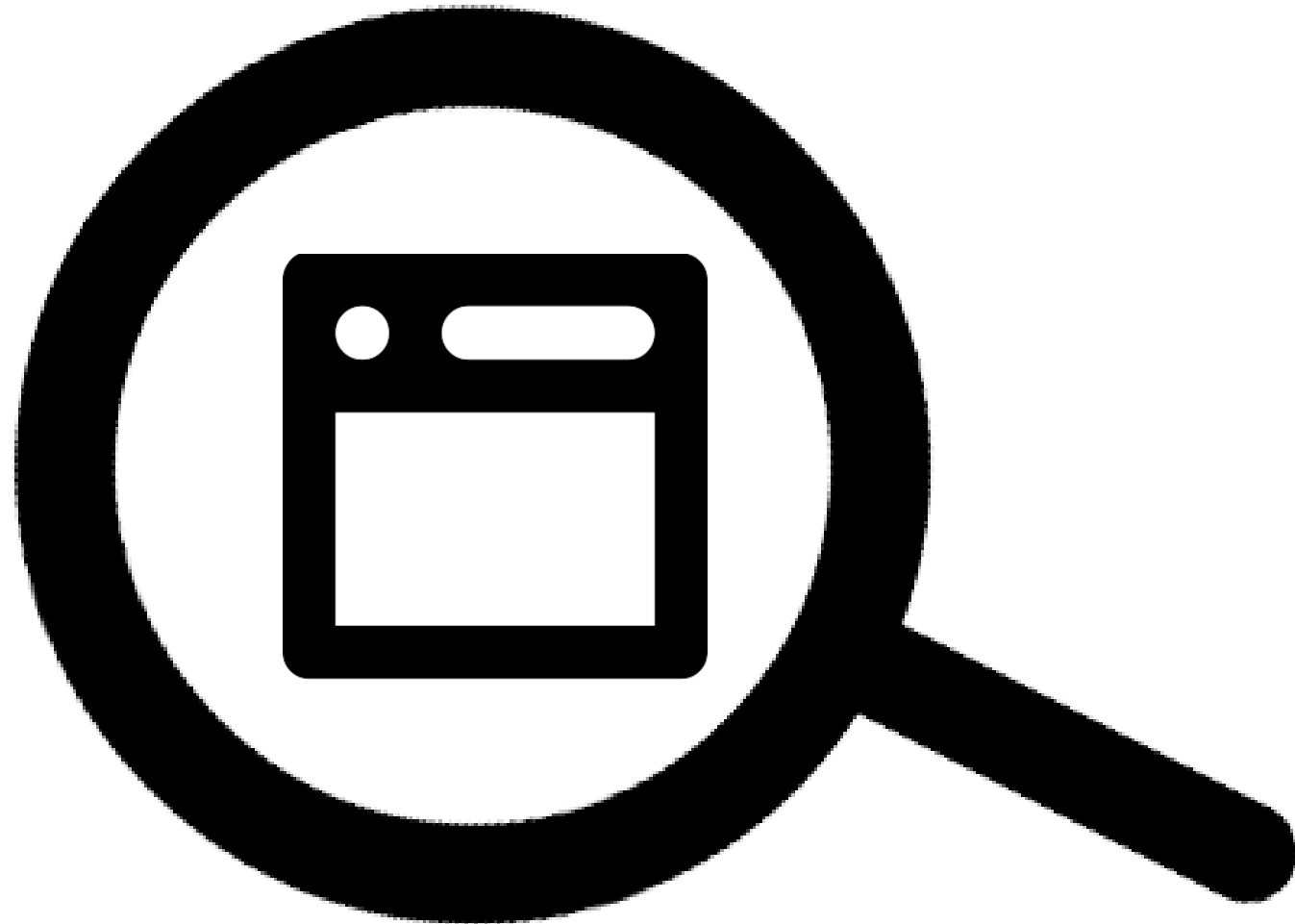


CARGAR DATOS



QUERY DATA

¿Cómo se usa Neo4j? – Interfaz con el motor de la base de datos





Neo4j 2.3.0



Node labels

* Category Customer
Employee Order Product
Supplier

Relationship types

* PART_OF PRODUCT
PURCHASED REPORTS_TO SOLD
SUPPLIES

Property keys

_GA_TIMER_MODULE_NR address
categoryID categoryName city
code companyName contactName
contactTitle country created_at
customerID customerName
description discontinued distance
employeeID favorites fax
firstName followers following
homePage id id_str lastName
location method name nodeRank
orderId phone picture
postalCode productID
productName profile_image_url



\$



```
S MATCH (e:Employee)-[:REPORTS_TO]-(sub:Employee) OPTIONAL MATCH (e)-[j-(o:...]
```



Graph



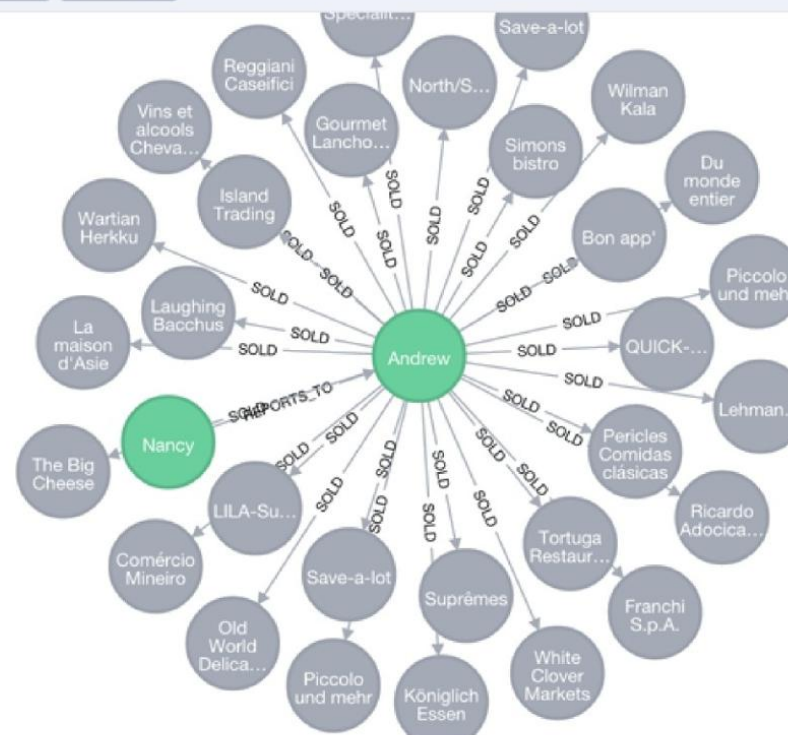
Rows



Code

*(32) Employee(2) Order(30)

*(31) REPORTS_TO(1) SOLD(30)



Displaying 32 nodes. 31 relationships (completed with 31 additional relationships).

AUTO-COMPLETE

ON



Graph

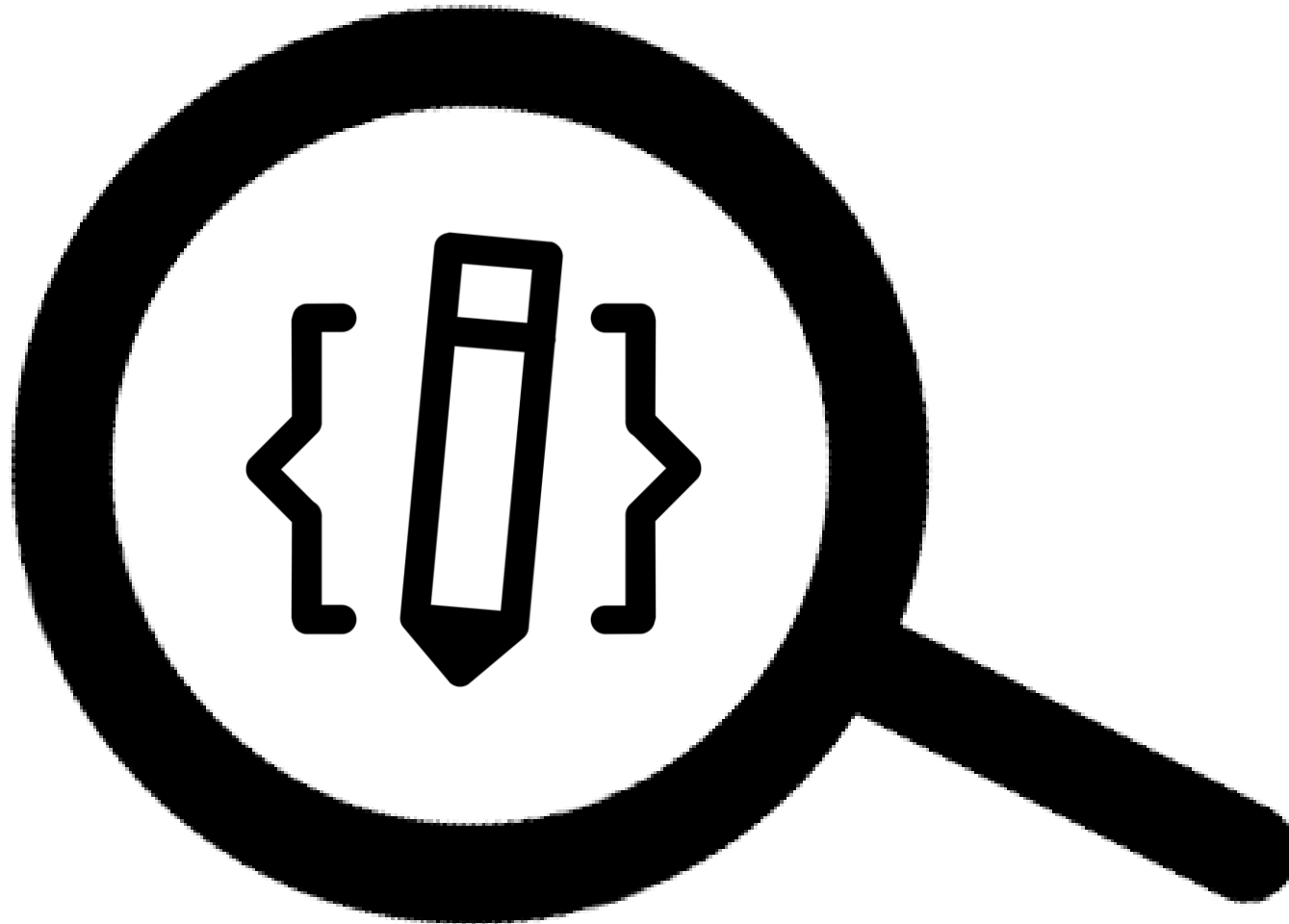
```
S MATCH (e:Employee)-[:REPORTS_TO]-(sub:Employee) OPTIONAL MATCH (e)-[j-(o:...]
```



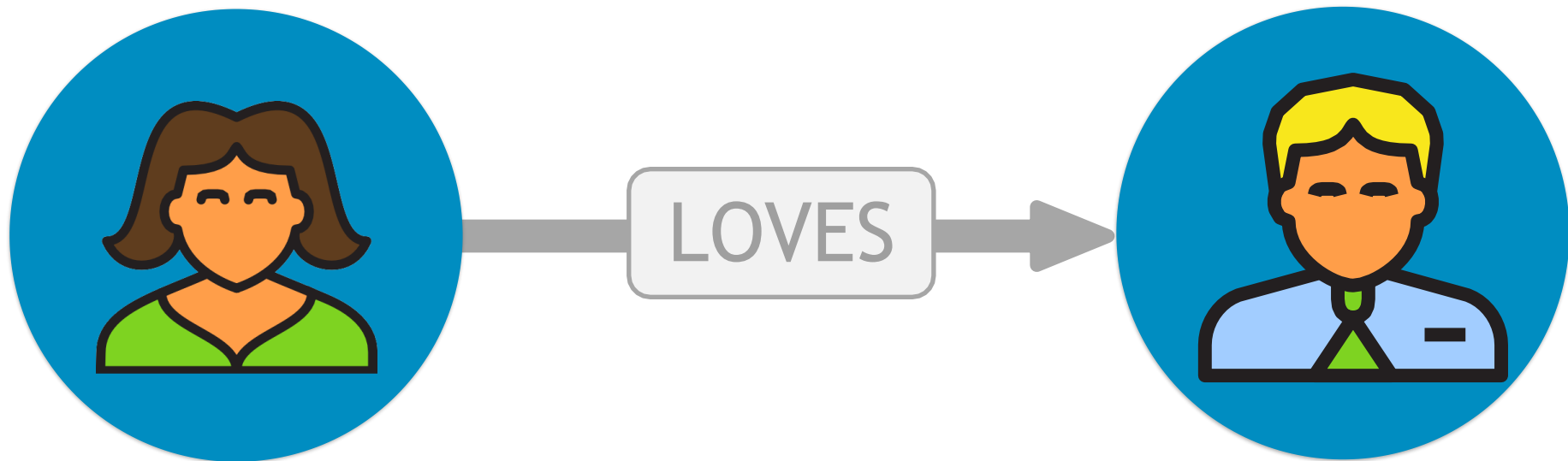
*(22) Employee(2) Order(20)

*(21) REPORTS_TO(1) SOLD(20)

¿Cómo se usa Neo4j? – Lenguaje de programación



Cypher - SQL for graphs – Creando datos



NODE

NODE

```
CREATE (:Person { name:"Ann" }) - [:LOVES]-> (:Person { name:"Dan" })
```

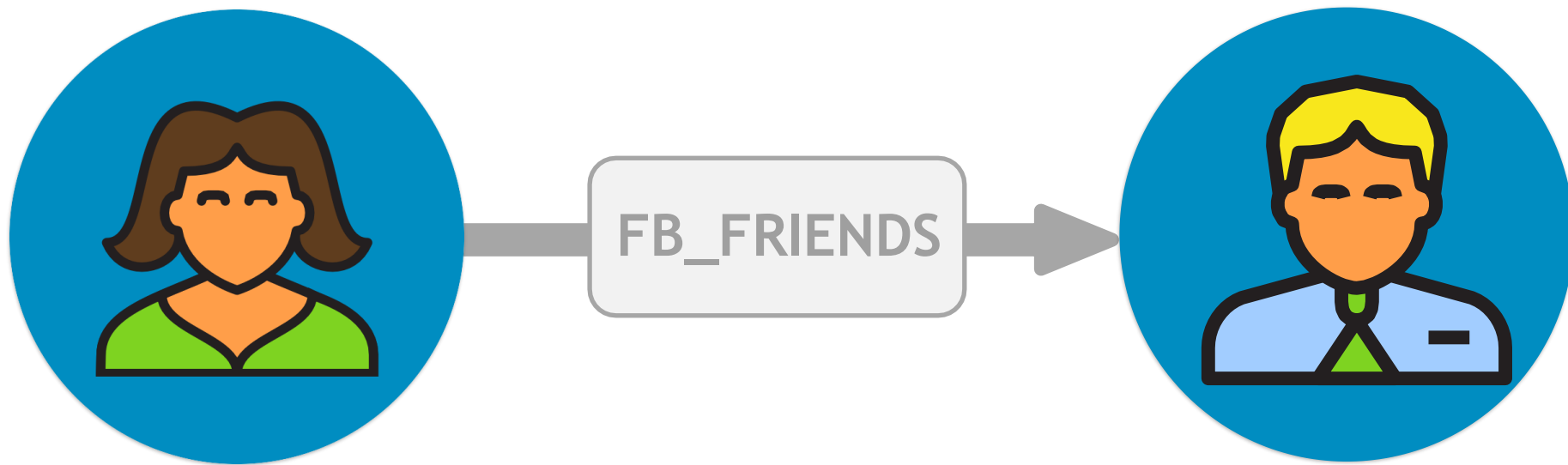
LABEL

PROPERTY

LABEL

PROPERTY

Cypher - SQL for graphs – Representación de la bidireccionalidad



`MATCH (:Person { name:"Ann" } ) - [:FB_FRIENDS] -> (:Person { name:"Dan" } )`

`MATCH (:Person { name:"Ann" } ) - [:FB_FRIENDS] - (:Person { name:"Dan" } )`

Cypher

Típica unión SQL compleja

```
(SELECT T.directReportees AS directReportees, sum(T.count) AS count
FROM (
  SELECT manager.pid AS directReportees, 0 AS count
  FROM person_reportee manager
  WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
UNION
  SELECT manager.pid AS directReportees, count(manager.directly_manages) AS count
  FROM person_reportee manager
  WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
GROUP BY directReportees
UNION
  SELECT manager.pid AS directReportees, count(reportee.directly_manages) AS count
  FROM person_reportee manager
  JOIN person_reportee reportee
  ON manager.directly_manages = reportee.pid
  WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
GROUP BY directReportees
UNION
  SELECT manager.pid AS directReportees, count(L2Reportees.directly_manages) AS count
  FROM person_reportee manager
  JOIN person_reportee L1Reportees
  ON manager.directly_manages = L1Reportees.pid
  JOIN person_reportee L2Reportees
  ON L1Reportees.directly_manages = L2Reportees.pid
  WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
GROUP BY directReportees
) AS T
GROUP BY directReportees)
UNION
(SELECT T.directReportees AS directReportees, sum(T.count) AS count
FROM (
  SELECT manager.directly_manages AS directReportees, 0 AS count
  FROM person_reportee manager
  WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
UNION
  SELECT reportee.pid AS directReportees, count(reportee.directly_manages) AS count
  FROM person_reportee manager
  JOIN person_reportee reportee
  ON manager.directly_manages = reportee.pid
  WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
GROUP BY directReportees
) AS T
GROUP BY directReportees)
UNION
(SELECT L2Reportees.directly_manages AS directReportees, 0 AS count
FROM person_reportee manager
JOIN person_reportee L1Reportees
ON manager.directly_manages = L1Reportees.pid
JOIN person_reportee L2Reportees
ON L1Reportees.directly_manages = L2Reportees.pid
WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
GROUP BY directReportees
) AS T
GROUP BY directReportees)
UNION
(SELECT L2Reportees.directly_manages AS directReportees, 0 AS count
FROM person_reportee manager
JOIN person_reportee L1Reportees
ON manager.directly_manages = L1Reportees.pid
JOIN person_reportee L2Reportees
ON L1Reportees.directly_manages = L2Reportees.pid
WHERE manager.pid = (SELECT id FROM person WHERE name = "fName lName")
GROUP BY directReportees
) AS T
GROUP BY directReportees)
```

La misma consulta usando Cypher

```
MATCH (boss) -[:MANAGES*0..3]->(sub),
        (sub) -[:MANAGES*1..3]->(report)
WHERE boss.name = "John Doe"
RETURN sub.name AS Subordinate,
        count(report) AS Total
```

Impacto del proyecto

Menos tiempo escribiendo consultas

- Más tiempo para entender las respuestas,
- dejar tiempo para hacer la siguiente pregunta

Menos tiempo depurando consultas:

- Más tiempo escribiendo el siguiente
- código Mejora de la calidad general de la base de código

Código más fácil de leer:

- Aceleración de la puesta en marcha para
- nuevos miembros del proyecto Mejora de la mantenibilidad y resolución de problemas

Neo4j y Cypher



Property graph

Nodos y relaciones, ambos con etiquetas y propiedades (clave-valor).



Cypher

Lenguaje de consulta declarativo que dibuja el patrón: (a)-[:REL]->(b).



Recorrido rápido

Las relaciones son punteros directos: multi-hop sin JOINS costosos.



Se corre con Docker

neo4j:latest en un contenedor; UI en :7474, driver bolt en :7687.

La consulta multi-hop: SQL vs Cypher

"¿Con qué actores actuó Bob?" — la misma pregunta en una base relacional y en un grafo.

SQL (relacional)

```
SELECT DISTINCT P2.name
FROM Persons P1
JOIN Relationships R1 ON R1.person_id=P1.id
JOIN Movies M ON M.id=R1.movie_id
JOIN Relationships R2 ON R2.movie_id=M.id
JOIN Persons P2 ON P2.id=R2.person_id
WHERE P1.name='Bob' AND P2.name<>'Bob';
-- 4 JOINS para 2 saltos
```

Cypher (grafo)

```
MATCH (bob:Person {name:'Bob'})
      -[:ACTED_IN]->(:Movie)
      <-[:ACTED_IN]-(co:Person)
WHERE co.name <> 'Bob'
RETURN DISTINCT co.name;

// el patrón ES la consulta
```

¿Cuándo conviene cada una?



Grafo (Neo4j)

- Relaciones profundas y de muchos saltos
- El dominio ES una red (linaje, recomendación, fraude)
- Consultas de caminos y conexiones
- Esquema flexible que evoluciona



Relacional (SQL)

- Datos tabulares y transaccionales
- Agregaciones sobre muchas filas
- Esquema estable y bien definido
- Integridad y reportes clásicos

No es 'uno u otro': muchas plataformas usan ambos, cada uno para lo que hace bien.

GRAFOS DE LINAJE · EL PUENTE CON MLOPS

Datos, features, experimentos, modelos y despliegues son nodos; "se derivó de", "entrenó", "se desplegó" son aristas.



Guardado en un grafo (Neo4j), permite preguntar por trazabilidad, impacto y auditoría.

Para qué sirve en producción



Trazabilidad

¿Qué datos y qué código produjeron ESTA predicción? Reproducir y explicar.



Análisis de impacto

Si cambio este dataset o feature, ¿qué modelos y servicios se ven afectados?

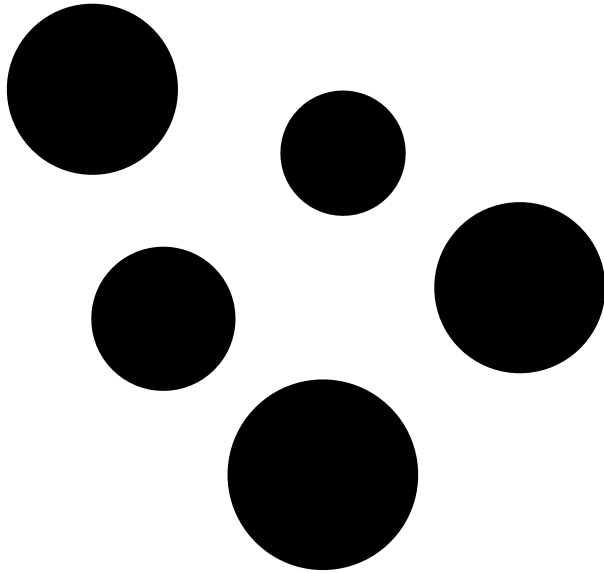


Auditoría y gobernanza

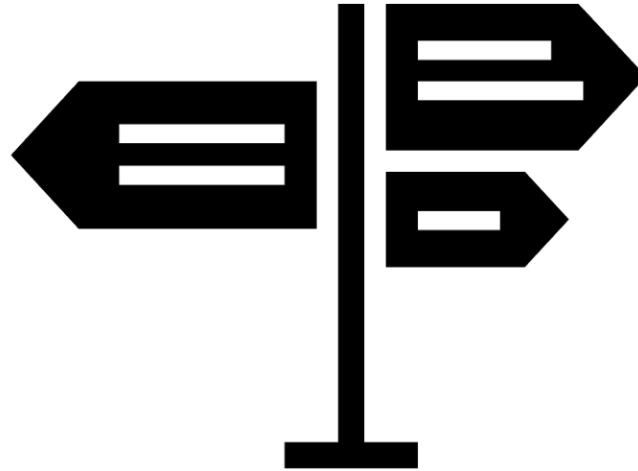
Evidencia para cumplimiento: quién entrenó qué, con qué datos y cuándo.

Lo conectamos con la gobernanza de la Sesión 7. Hoy modelamos el linaje como grafo y lo consultamos.

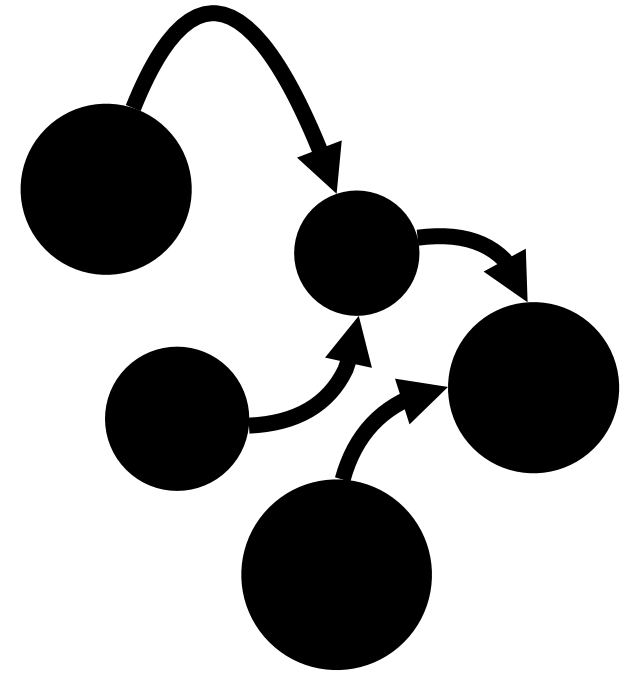
3 pasos para crear el grafo



Importar nodos

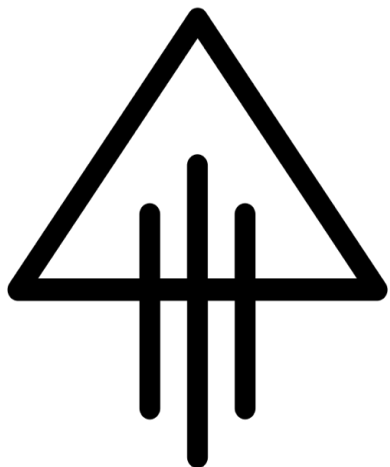


Crear indices

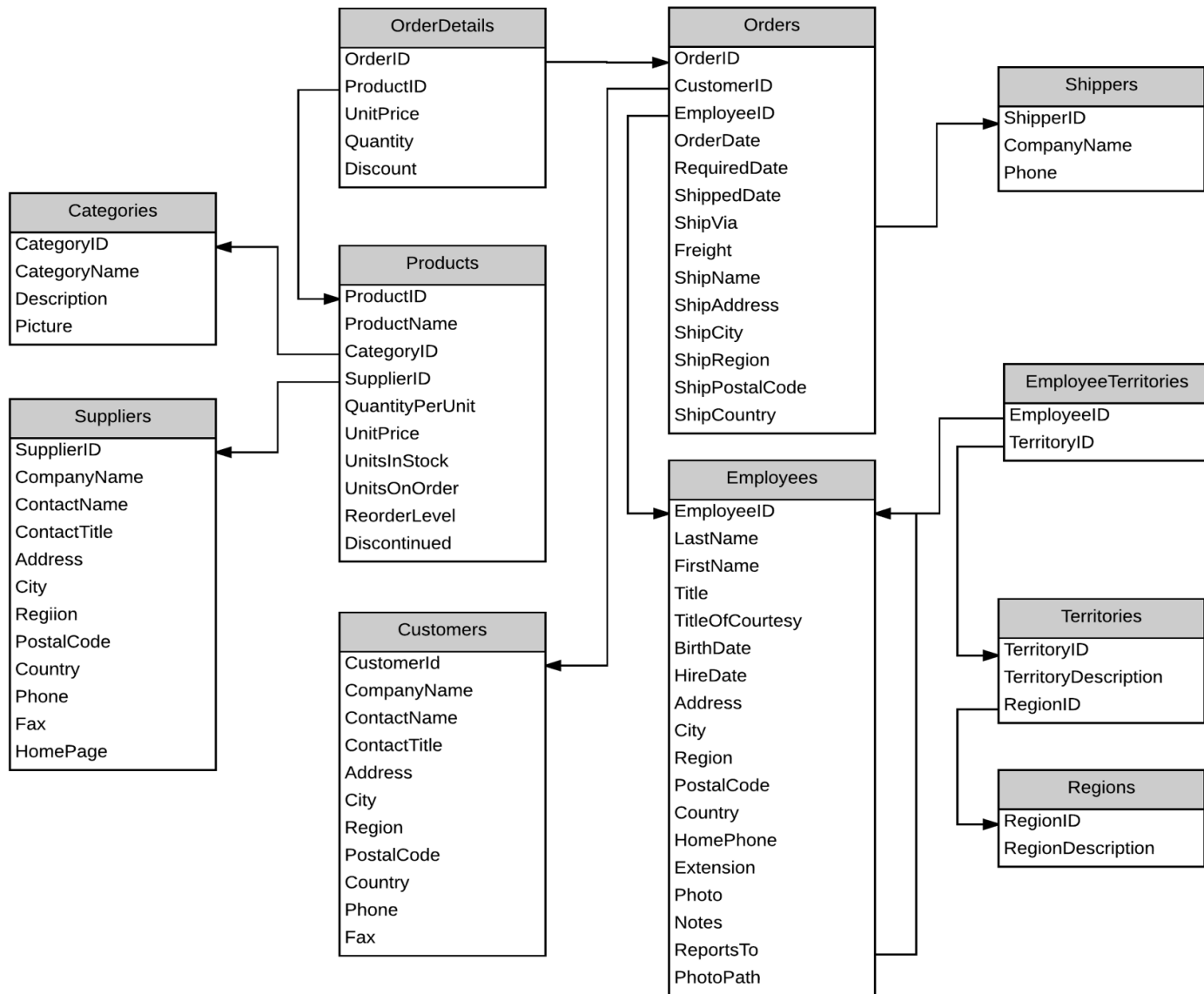


Importar relaciones

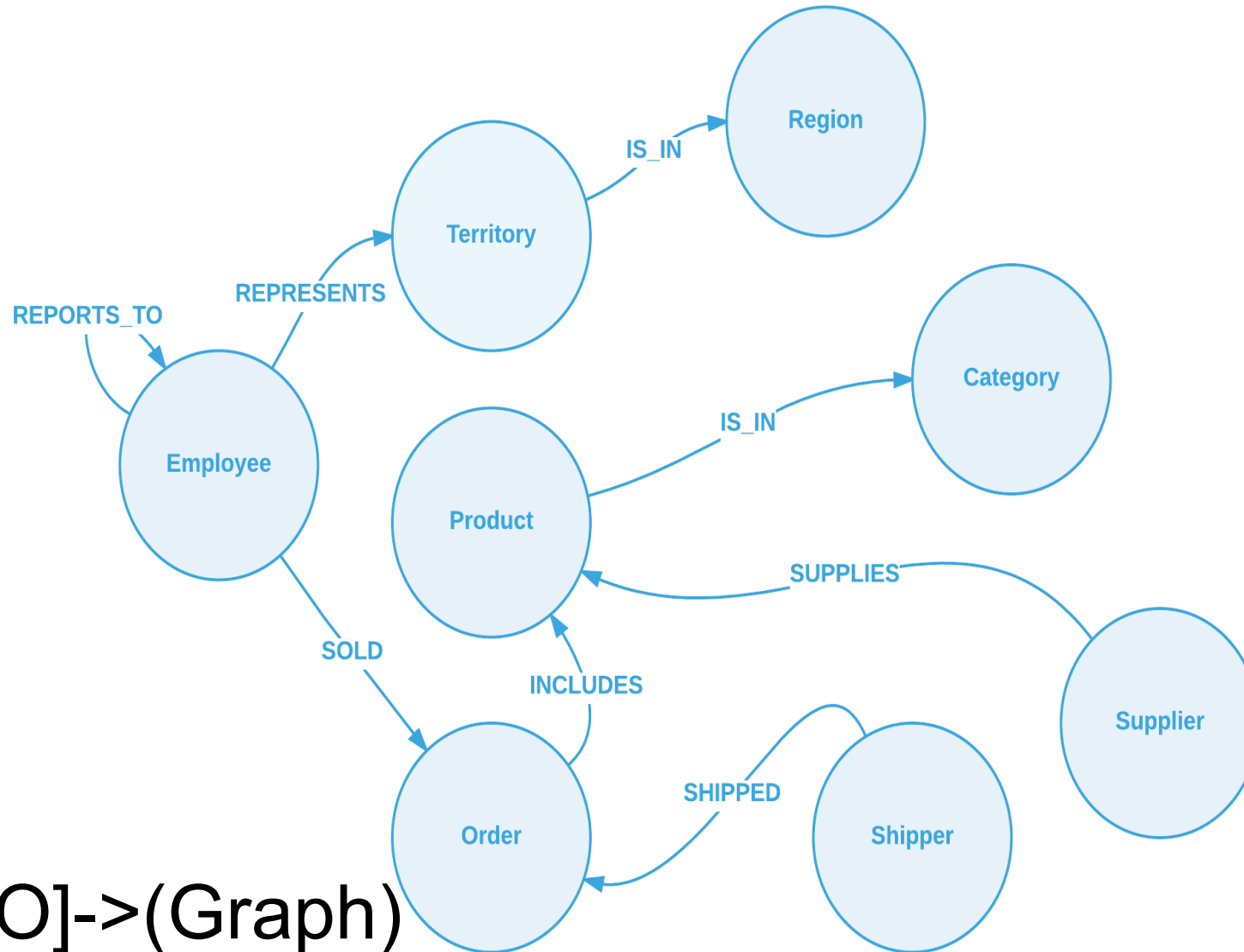
Cypher – Comparación con SQL



Northwind

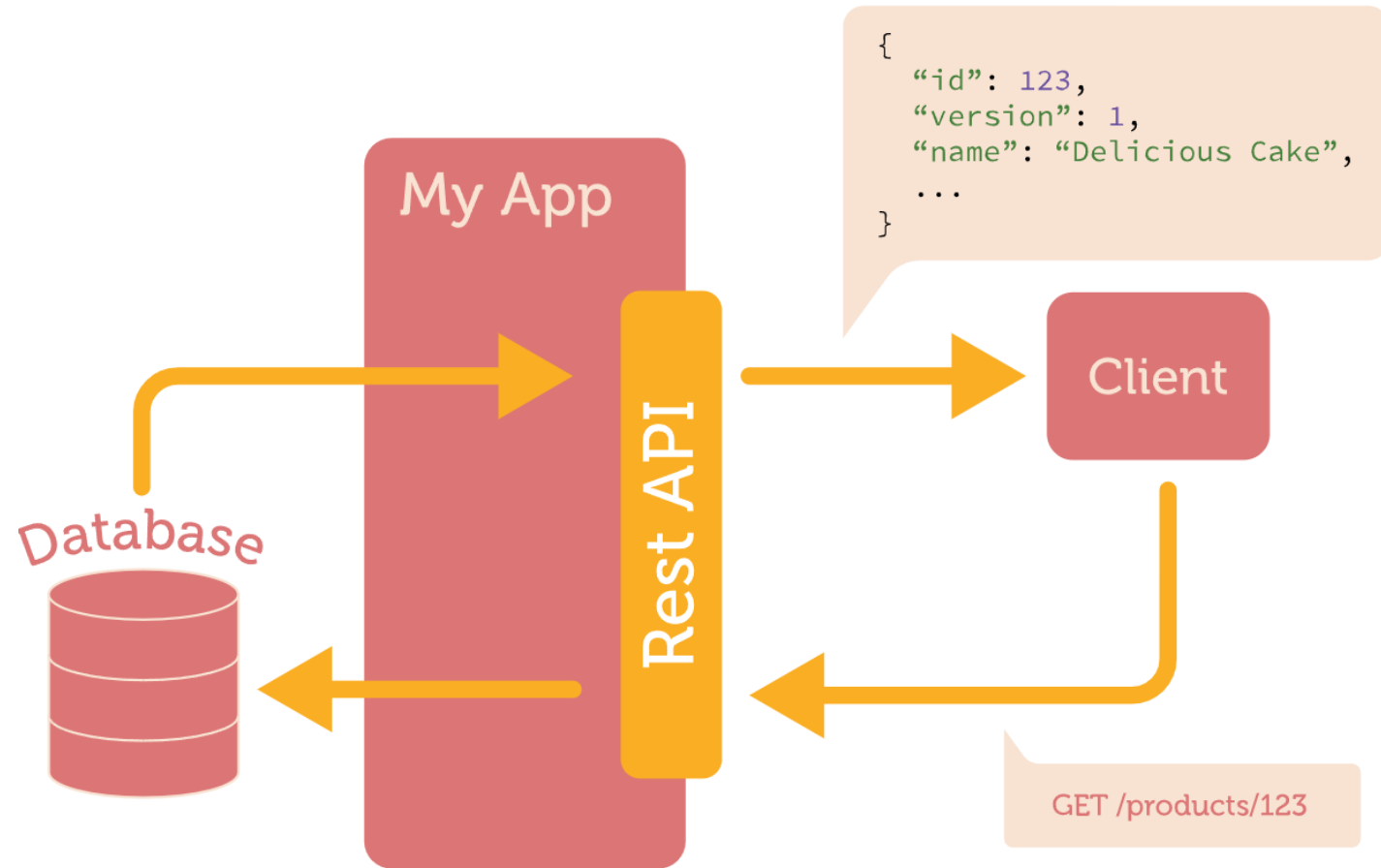


Cypher – Comparación con SQL



()-[:TO]->(Graph)

REST API



El problema que REST no resuelve bien



Over-fetching

El endpoint REST devuelve un objeto fijo y grande; el cliente sólo necesitaba dos campos y recibe todo.

Under-fetching

Para armar una vista necesitas datos de varios endpoints → varias llamadas (N+1) y más latencia.

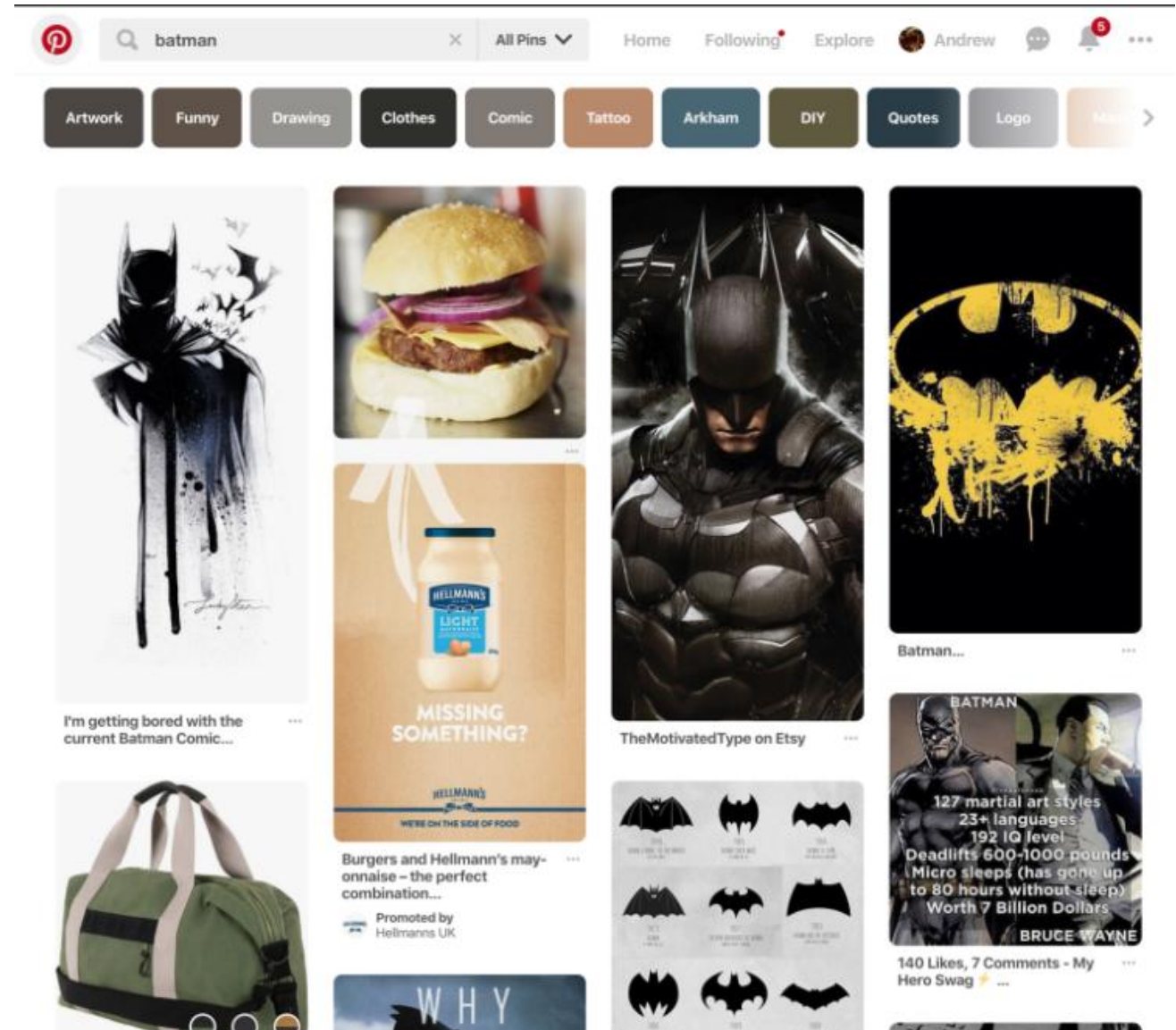
Con GraphQL

Una sola petición, a un solo endpoint, pidiendo exactamente los campos que necesitas — ni más ni menos.

```
query {
  model(id: "churn@3") {
    name version
    metrics { auc }
  }
} # solo eso vuelve
```

Problemas?

- Quiero obtener el nombre y descripción de cada imagen de una REST API de un website.

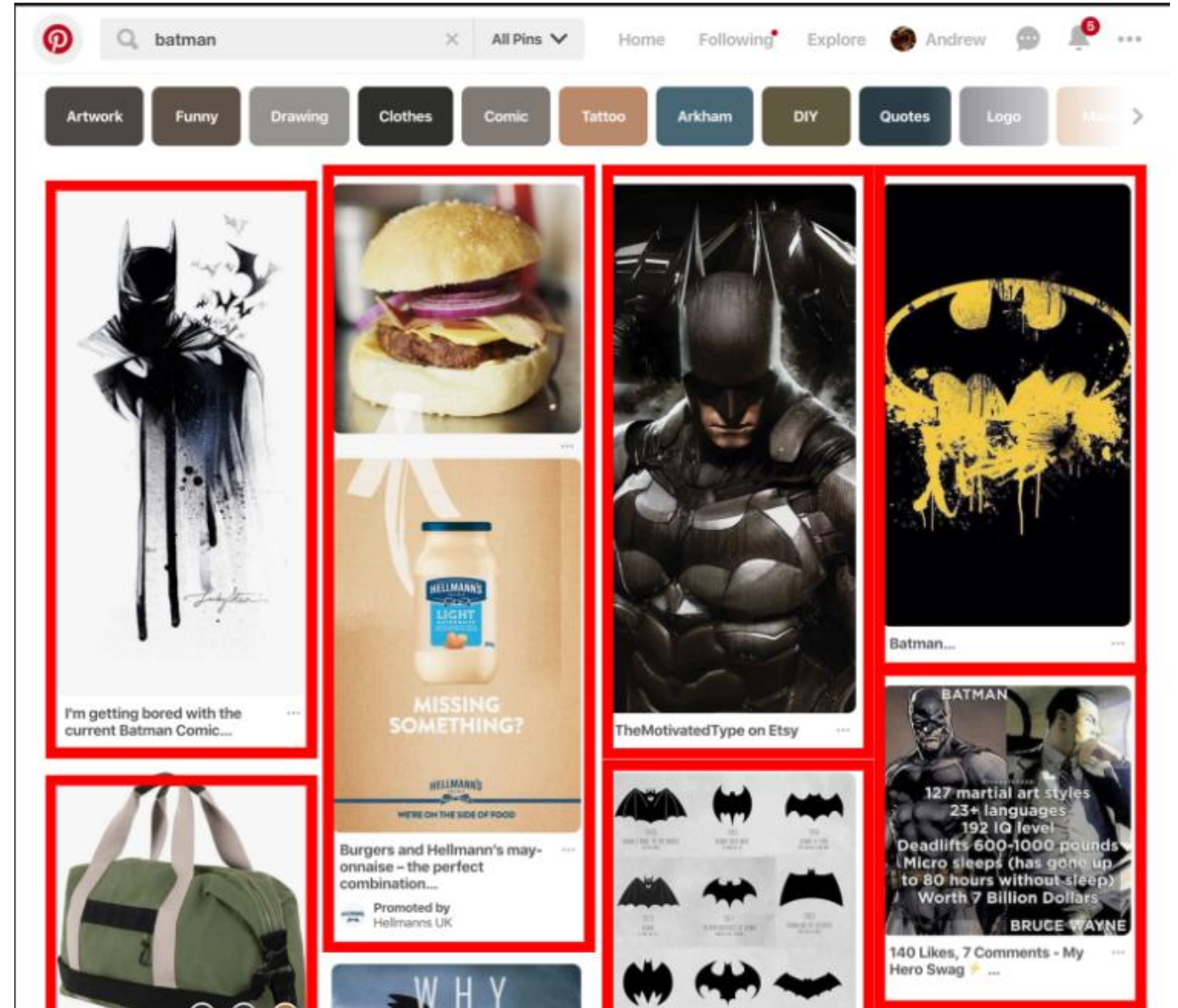


Problemas?

- Una llamada a la API para la recopilación
- Luego, una llamada por artículo para obtener más detalles

```
GET /results
GET /results/1234
GET /results/5678
GET /results/9012
```

$$O = n + 1$$



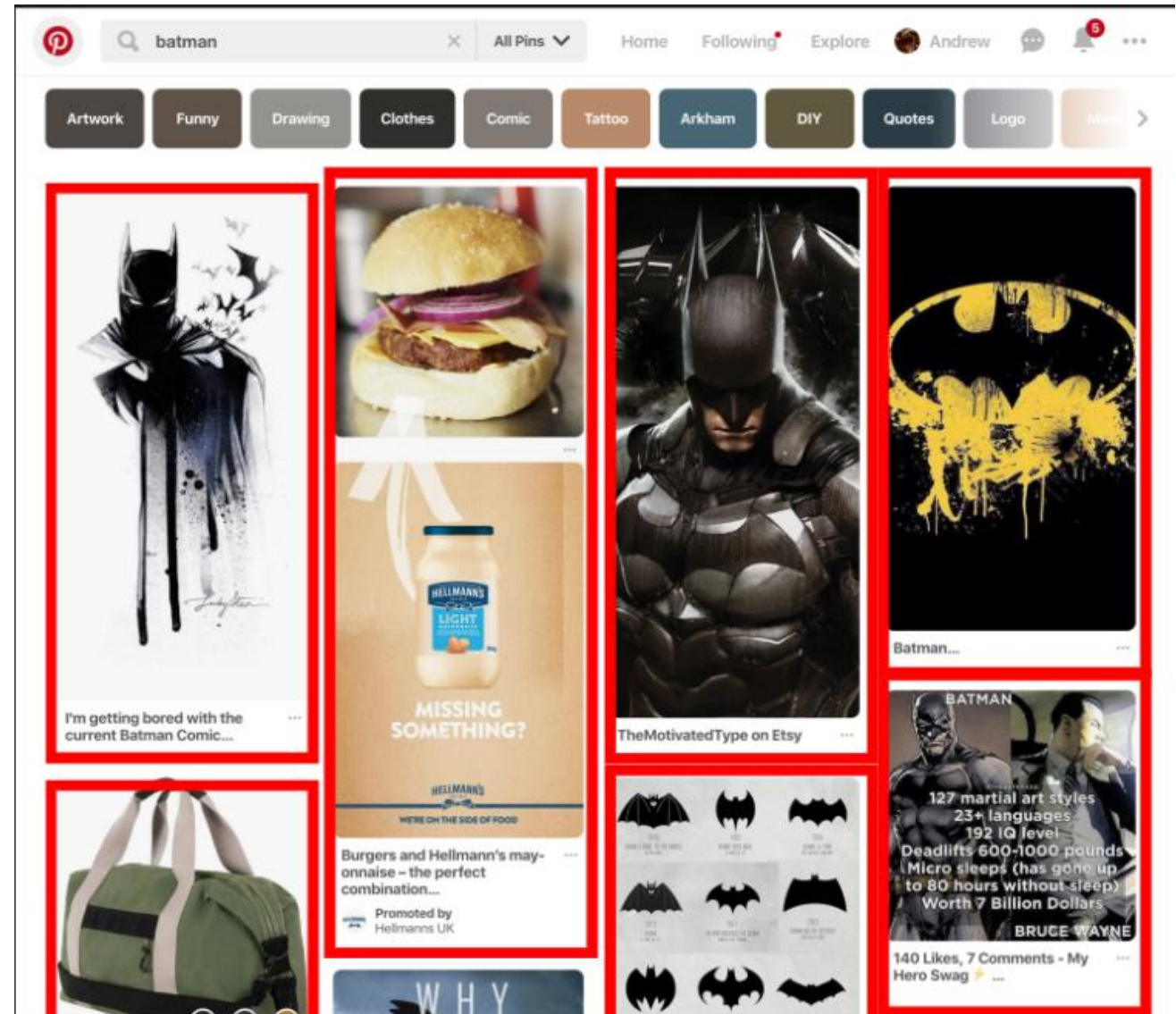
Problemas?

- GraphQL puede agrupar muchos datos en una sola consulta en lugar de muchas

$O = 1$

```
POST /graphql

{
  pins(last: 10) {
    imageUrl
    name
    likes
  }
}
```



El problema N+1: muchas llamadas vs una

Armar una vista (perfil: usuario + sus posts + sus seguidores) en REST suele requerir una llamada para la lista y una por cada elemento.

REST

```
GET /users/1
GET /users/1/posts
GET /users/1/followers ...
```

$O = n + 1$ llamadas

GraphQL

```
query { user(id:1){
  posts{title} followers{name}
} }
```

$O = 1$ llamada

GraphQL empaqueta muchos datos en una sola consulta.

¿Qué es GraphQL?

Un lenguaje de consulta para APIs y un runtime para resolverlas. El servidor publica un esquema tipado; el cliente pide una forma exacta de datos a un único endpoint.



Trae su documentación integrada (GraphiQL): el cliente explora el esquema sin herramientas externas.

Conceptos clave



Schema / SDL

El contrato tipado: qué tipos y campos existen (Book, Model, Metric).



Query

Lectura: pides los campos que quieres. No hay over-fetching.



Mutation

Escritura: crear, actualizar o borrar (createBook, deleteBook).



Subscription

Datos en tiempo real vía suscripción (eventos, streaming).



Resolver

La función que produce cada campo: de memoria, de una DB, de otro servicio.

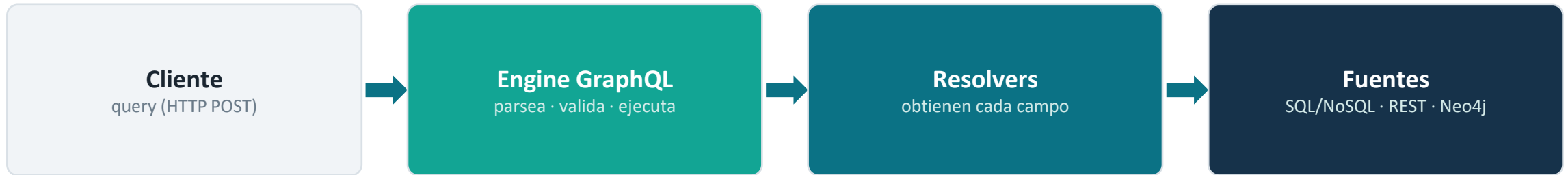


Tipado fuerte

Entrada y salida validadas por el esquema; errores claros.

Arquitectura y flujo de datos

El servidor GraphQL se sitúa entre los clientes y las fuentes de datos. Recibe la query, la valida y ejecuta los resolvers.



La respuesta se ensambla en un JSON con la misma forma que la query, y vuelve al cliente.

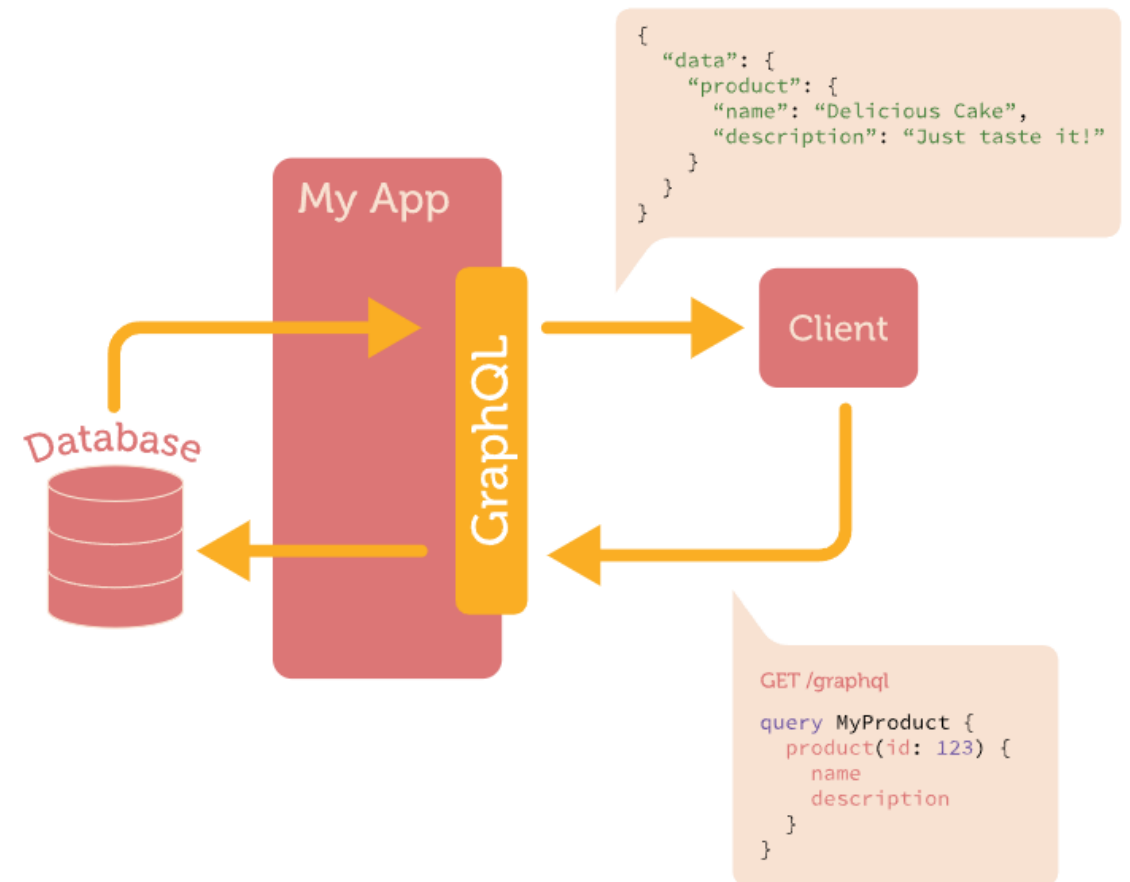
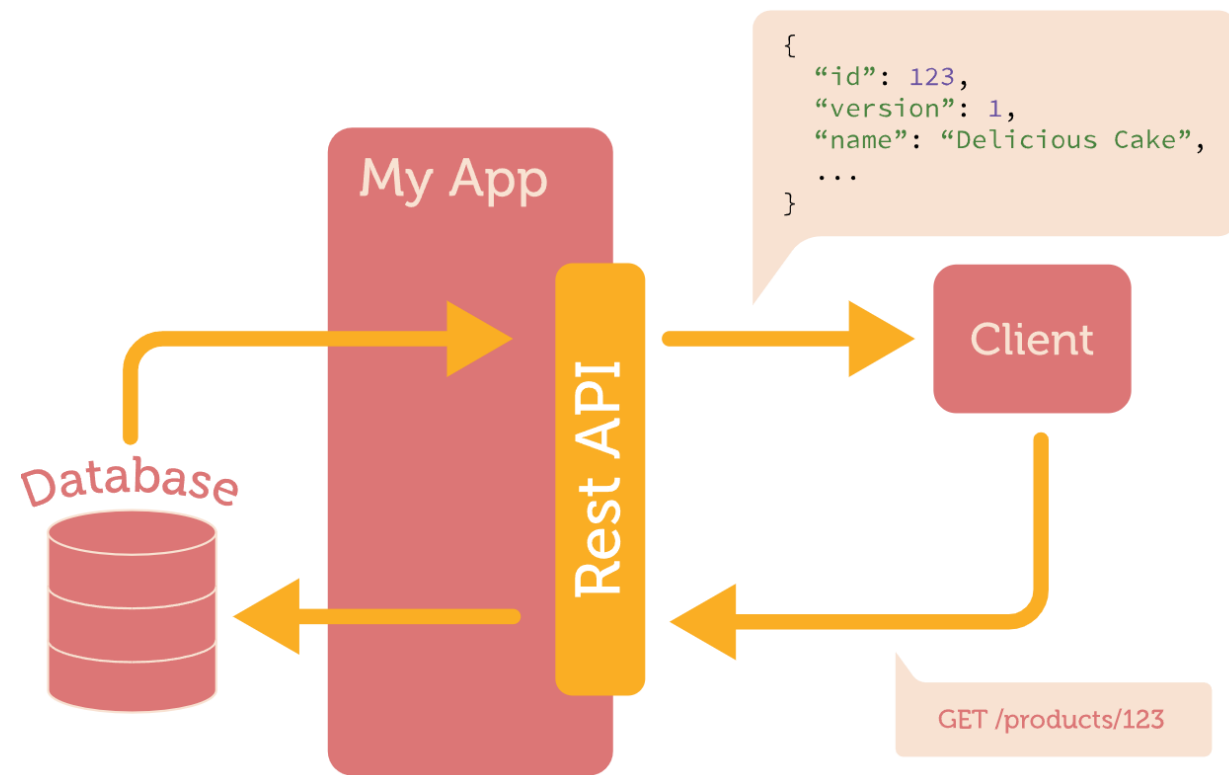
El esquema

El Esquema es el contrato formal y fuertemente tipado entre el cliente y el servidor, escrito en SDL (Schema Definition Language).

```
type Query {  
  articulo(id: ID!): Artículo  
  articulos(limit: Int = 10): [Artículo!]!  
}  
  
type Artículo {  
  id: ID!  
  titulo: String!  
  contenido: String  
  autor: Usuario!  
  comentarios: [Comentario!]  
}
```

```
type Usuario {  
  id: ID!  
  nombre: String!  
  email: String!  
}  
  
type Comentario {  
  id: ID!  
  texto: String!  
  autor: Usuario!  
}
```

El esquema



[Server]
API shape;
**GraphQL
Schema**

[Client]
Query;
GraphQL

[Client]
Result

The screenshot shows the GraphQL.org website with a dark background and a network-like pattern. The browser's address bar shows 'graphql.org'. The navigation bar includes links for 'Learn', 'Code', 'Community', 'Blog', and 'Spec', along with a search bar labeled 'Search docs...'. The main content area is divided into three sections by vertical green lines, each with a title and a code snippet. The first section, 'Describe your data', shows a GraphQL schema for a 'Project' type. The second section, 'Ask for what you want', shows a query for the 'GraphQL' project. The third section, 'Get predictable results', shows the JSON response of the query. At the bottom, there are two buttons: 'Get Started' and 'Learn More'.

GraphQL | A query language for APIs

graphql.org

Learn Code Community Blog Spec

Search docs...

GraphQL

Describe your data

```
type Project {  
  name: String!  
  tagline: String!  
  contributors: [User]!  
}
```

Ask for what you want

```
{  
  project(name: "GraphQL") {  
    tagline  
  }  
}
```

Get predictable results

```
{  
  "project": {  
    "tagline": "A query language for APIs"  
  }  
}
```

Get Started Learn More

GraphQL vs REST, sin exageraciones

Aspecto	REST	GraphQL
Endpoints	Muchos (uno por recurso)	Uno solo (/graphql)
Forma de la respuesta	Fija por endpoint	La define el cliente
Over/under-fetching	Frecuente	Se evita
Documentación	Externa (OpenAPI/Swagger)	Integrada en el esquema
Caching HTTP	Simple y maduro	Más complejo
Curva / complejidad	Baja	Media (esquema, N+1)

Regla práctica: REST para el borde simple y cacheable; GraphQL cuando el cliente necesita flexibilidad y agregación.

Partes fundamentales de un servidor GraphQL

Código de servidor GraphQL ejecutable

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');

const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});

const query = `query { serverTime }`;

graphql(schema, query).then(result => {
  console.log(result.data);
});

// Prints:
// {serverTime: "9/5/2016, 6:28:46 PM"}
```

```
import graphene
from datetime import datetime

class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

schema = graphene.Schema(query=Query)

query = 'query { serverTime }'

result = schema.execute(query)
print(result.data)

#: Prints: OrderedDict([('serverTime', '2017-07-24
19:10:38.127089')])
```

Código de servidor GraphQL ejecutable / Definición de API (esquema GraphQL)

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');
```

```
const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});
```

```
const query = `query { serverTime }`;

graphql(schema, query).then(result =>{
  console.log(result.data);
});

// Prints:
// {serverTime: "9/5/2016, 6:28:46 PM"}
```

```
import graphene
from datetime import datetime
```

```
class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

schema = graphene.Schema(query=Query)
```

```
query = 'query { serverTime }'

result = schema.execute(query)
print(result.data)

#: Prints: OrderedDict([('serverTime', '2017-07-24
19:10:38.127089')])
```


Código de servidor GraphQL ejecutable / Query en GraphQL

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');

const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});

const query = `query { serverTime }`;

graphql(schema, query).then(result => {
  console.log(result.data);
});

// Prints:
// {serverTime: "9/5/2016, 6:28:46 PM"}
```

```
import graphene
from datetime import datetime

class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

schema = graphene.Schema(query=Query)

query = 'query { serverTime }'

result = schema.execute(query)
print(result.data)

#: Prints: OrderedDict([('serverTime', '2017-07-24
19:10:38.127089')])
```

Código de servidor GraphQL ejecutable / Ejecución del Query (query + schema → result)

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');

const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});

const query = `query { serverTime }`;

graphql(schema, query).then(result => {
  console.log(result.data);
});

// Prints:
// {serverTime: "9/5/2016, 6:28:46 PM"}
```

```
import graphene
from datetime import datetime

class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

schema = graphene.Schema(query=Query)

query = 'query { serverTime }'

result = schema.execute(query)
print(result.data)

#: Prints: OrderedDict([('serverTime', '2017-07-24
19:10:38.127089')])
```

Código de servidor GraphQL ejecutable / resultado

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');

const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});

const query = `query { serverTime }`;

graphql(schema, query).then(result =>{
  console.log(result.data);
});

// Prints:
// {serverTime: "9/5/2016, 6:28:46 PM"}
```

```
import graphene
from datetime import datetime

class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

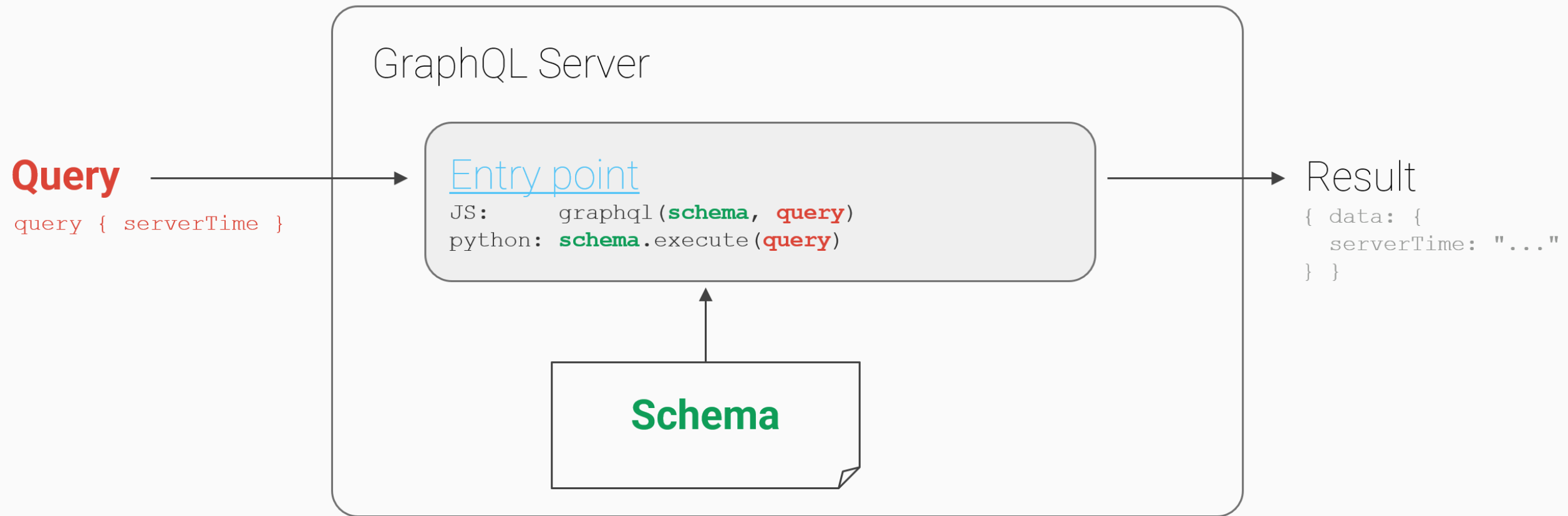
schema = graphene.Schema(query=Query)

query = 'query { serverTime }'

result = schema.execute(query)
print(result.data)

#: Prints: OrderedDict([('serverTime', '2017-07-24
19:10:38.127089')])
```

Partes del server GraphQL



Servidor GraphQL usando HTTP

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');

const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});

const query = `query { serverTime }`;

graphql(schema, query).then(result => {
  console.log(result.data);
});
```

```
import graphene
from datetime import datetime

class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

schema = graphene.Schema(query=Query)

query = `query { serverTime }`

result = schema.execute(query)
print(result.data)
```

Servidor GraphQL usando HTTP

```
const {
  graphql, GraphQLSchema, GraphQLObjectType,
  GraphQLString,
} = require('graphql');

const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: 'Query',
    fields: {
      serverTime: {
        type: GraphQLString,
        resolve() {
          return (new Date).toLocaleString();
        },
      },
    },
  }),
});

app.post('/graphql', (req, res) => {
  graphql(schema, req.body).then(res.json);
});
```

```
import graphene
from datetime import datetime

class Query(graphene.ObjectType):
    server_time = graphene.Field(graphene.String)

    def resolve_server_time(obj, args, context, info):
        return str(datetime.now())

schema = graphene.Schema(query=Query)

@app.route('/graphql')
def graphql():
    result = schema.execute(request.body)
    return json.dumps({
        data: result.data,
        errors: result.errors
    })
```

Query y Esquema

```
query {  
  serverTime  
}
```

```
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: 'Query',  
    fields: {  
      serverTime: {  
        type: GraphQLString,  
        resolve() { /* ... */ },  
      },  
    },  
  }),  
})
```

```
class Query(graphene.ObjectType):  
    server_time = graphene.Field(...)  
  
    def resolve_server_time(obj, args,  
context, info):  
    # ...  
  
schema = graphene.Schema(query=Query)
```

Object Type

```
query {  
  serverTime  
}
```

```
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: 'Query',  
    fields: {  
      serverTime: {  
        type: GraphQLString,  
        resolve() { /* ... */ },  
      },  
    },  
  }),  
});
```

```
class Query(graphene.ObjectType):  
    server_time = graphene.Field(...)  
  
    def resolve_server_time(obj, args,  
context, info):  
    # ...  
  
schema = graphene.Schema(query=Query)
```


Object Type/fields

```
query {  
  serverTime  
}
```

```
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: 'Query',  
    fields: {  
      serverTme: {  
        type: GraphQLString,  
        resolve() { /* ...*/ },  
      },  
    },  
  }),  
})
```

```
class Query(graphene.ObjectType):  
  server_time = graphene.Field(...)  
  
  def resolve_server_time(obj, args,  
    context, info):  
    # ...  
  
schema = graphene.Schema(query=Query)
```

Object Type/fields/resolvers

```
query {  
  serverTime  
}
```

```
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: 'Query',  
    fields: {  
      serverTime: {  
        type: GraphQLString,  
        resolve() { /* ... */ },  
      },  
    },  
  }),  
})
```

```
class Query(graphene.ObjectType):  
    server_time = graphene.Field(...)
```

```
    def resolve_server_time(obj, args,  
context, info):  
    # ...
```

```
schema = graphene.Schema(query=Query)
```

Field con un Object Type

```
query {  
  serverTime { hour, minute, second }  
}
```

```
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: 'Query',  
    fields: {  
      serverTime: {  
        type: Time,  
        resolve() { /* ... */ },  
      },  
    },  
  },  
})
```

```
class Query(graphene.ObjectType):  
  server_time = graphene.Field(Time)  
  
  def resolve_server_time(obj, args,  
    context, info):  
    # ...  
  
schema = graphene.Schema(query=Query)
```

schema

```
const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: "Query",
    fields: {
      serverTime: {
        resolve() {
          /* ... */
        },
        type: new GraphQLObjectType({
          name: "Time",
          fields: {
            hour: {type: GraphQLInt},
            minute: {type: GraphQLInt},
            second: {type: GraphQLInt},
          },
        }),
      },
    },
  }),
});
```

query input

```
{
  query {
    serverTime {
      hour
      minute
      second
    }
  }
}
```

output

```
{
  data: {
    serverTime: {
      hour: 18,
      minute: 33,
      second: 58
    }
  }
}
```

Funciones para resolver un “field”

```
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: 'Query',  
    fields: {  
      serverTime: {  
        type: Time,  
        resolve() { /* ... */ },  
      },  
    },  
  })  
});
```

```
class Query(graphene.ObjectType):  
    serverTime = graphene.Field(Time)  
  
    def resolve_server_time(obj, args,  
context, info):  
    # ...  
  
schema = graphene.Schema(query=Query)
```

Funciones para resolver un “field”

```
const Time = new GraphQLObjectType({
  name: "Time",
  fields: {
    hour: {
      type: GraphQLInt,
      resolve: obj => obj.getHours() },
    minute: {
      type: GraphQLInt,
      resolve: obj => obj.getMinutes() },
    second: {
      type: GraphQLInt,
      resolve: obj => obj.getSeconds() }, }
});
```

```
const schema = new GraphQLSchema({
  query: new GraphQLObjectType({
    name: "Query",
    fields: {
      serverTime: {
        type: Time,
        resolve() { return new Date; }
      }
    }
  })
});
```

```
class Time(graphene.ObjectType):
    hour = graphene.Int()
    minute = graphene.Int()
    second = graphene.Int()

    def resolve_hour(obj, args, context, info):
        return obj.hour

    def resolve_minute(obj, args, context, info):
        return obj.minute

    def resolve_second(obj, args, context, info):
        return obj.second

class Query(graphene.ObjectType):
    server_time = graphene.Field(Time)

    def resolve_server_time(obj, args, context, info):
        return datetime.now()

schema = graphene.Schema(query=Query)
```

Resolvers triviales

```
someField(obj) {  
  return obj.someField;  
}
```

```
resolve_some_field(obj):  
  return obj.some_field
```

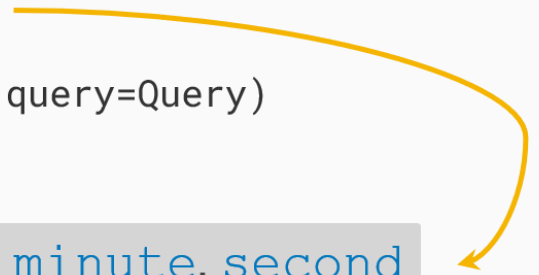
```
const Time = new GraphQLObjectType({  
  name: "Time",  
  fields: {  
    hour: {type: GraphQLInt},  
    minute: {type: GraphQLInt},  
    second: {type: GraphQLInt}}});  
  
const schema = new GraphQLSchema({  
  query: new GraphQLObjectType({  
    name: "Query",  
    fields: {  
      serverTime: {  
        type: Time,  
        resolve(){  
          const date = new Date;  
          return {  
            hour: date.getHours(),  
            minute: date.getMinutes(),  
            second: date.getSeconds(),  
          };  
        }  
      }  
    }  
  })  
});
```

```
class Time(graphene.ObjectType):  
    hour = graphene.Int()  
    minute = graphene.Int()  
    second = graphene.Int()
```

```
class Query(graphene.ObjectType):  
    server_time = graphene.Field(Time)  
  
    def resolve_server_time(obj, args, context, info):  
        return datetime.now()
```

```
schema = graphene.Schema(query=Query)
```

has property `hour, minute, second`



Mutaciones

```
query {  
  createUser(name: "John Doe") {  
    id  
  }  
}
```


Mutaciones

```
query {  
  user(name:"John Doe"){ id }  
  article(id:123) { text }  
}
```

} Run in parallel

```
mutation {  
  createUser(...) {...}  
  createArticle(...) {...}  
}
```

} Run in series

Mutaciones

```
const CreatePersonResult = new GraphQLObjectType({
  name: 'CreatePersonResult', fields: {
    ok: { type: GraphQLBoolean },
    person: { type: Person },
  }
})
```

```
const schema = new GraphQLSchema({
  query: ...,
  mutation: new GraphQLObjectType({
    name: 'Mutation',
    fields: {
      CreatePerson: {
        type: CreatePersonResult,
        args: { name: { type: GraphQLString } },
        resolve(obj, args) {
          const person = {name: args.name};
          // Should do something that persists
          // the person
          return { ok: true, person };
        }
      }
    }
  })
})
```

```
class CreatePerson(graphene.Mutation):
    class Input:
        name = graphene.Argument(graphene.String)

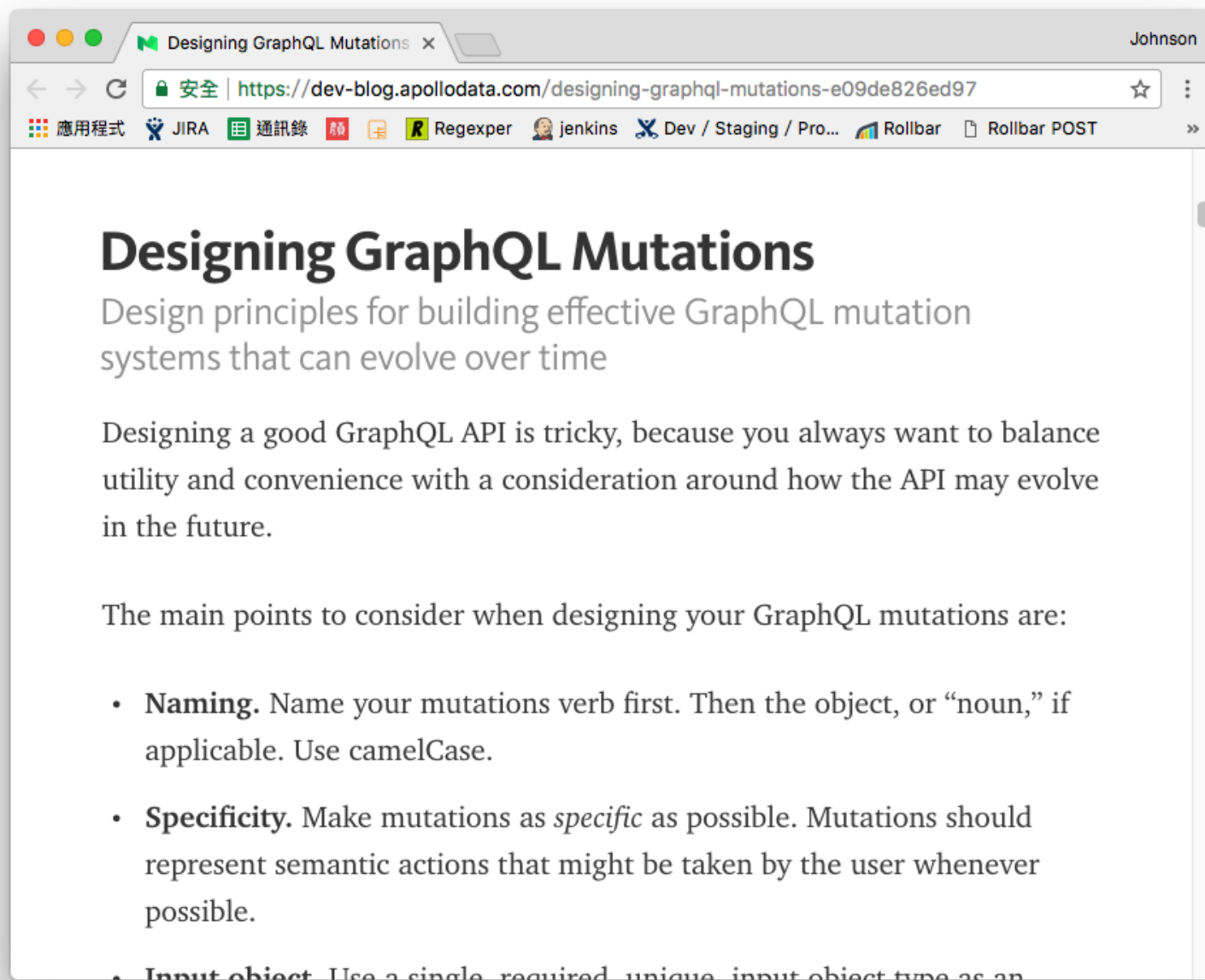
    ok = graphene.Field(graphene.Boolean)
    person = graphene.Field(lambda: Person)

    @staticmethod
    def mutate(root, args, context, info):
        #: Should do something that persists
        #: the new Person here
        person = Person(name=args.get('name'))
        ok = True
        return CreatePerson(person=person, ok=ok)
```

```
class Mutation(graphene.ObjectType):
    create_person = CreatePerson.Field()
```

```
schema = graphene.Schema(
    query=..., mutation=Mutation
)
```

Mutaciones



Hacer solicitudes a GraphQL Servers – via HTTP

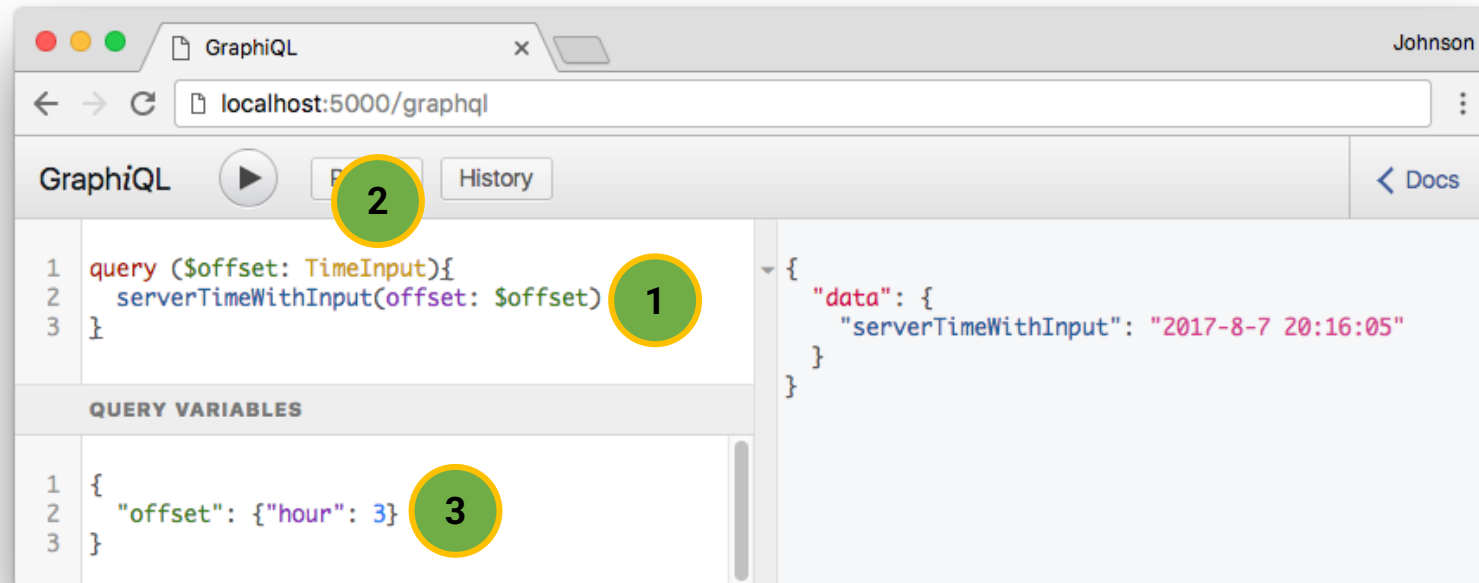
- Depende del servidor([apollo-server](#) / [Flask-GraphQL](#))
- Inspeccionar solicitudes de red (graphql)
- Principalmente acepta:

```
POST /graphql HTTP/1.1  
Content-Type: application/json
```

```
{  
  "query": "...GraphQL Query string...",  
}
```

Hacer solicitudes a GraphQL Servers – via HTTP

1. Sustituye el valor estático en la consulta por `$variableName`
2. Declarar `$variableName` como una de las variables aceptadas por la consulta
3. Pass `variableName: value` en el diccionario de variables separado, específico para transporte (normalmente JSON)



Hacer solicitudes a GraphQL Servers – via HTTP

1. Sustituye el valor estático en la consulta por `$variableName`
2. Declarar `$variableName` como una de las variables aceptadas por la consulta
3. Pass `variableName: value` en el diccionario de variables separado, específico para transporte (normalmente JSON)

```
POST /graphql HTTP/1.1
Content-Type: application/json
```

```
{
  "query":
    "query($offset:TimeInput){serverTimeWithInput(offset:$offset)}",
  "variables": {
    "offset": { "hour": 3 }
  }
}
```

Some old GraphQL servers only accept
"variables" as JSON **strings**

Hacer solicitudes a GraphQL Servers – Tecnicas avanzadas

- [Field alias](#) - renombrar clave objeto como resultado
- [Fragments](#) - para dedup fields
- [Inline Fragments](#) - para [union types](#)
- [Directives](#) - Consulta selectiva para campos

Lectura ampliada: [The Anatomy of GraphQL queries](#)

Ventajas... y sus costos

Ventajas

- Una request trae exactamente lo pedido
- Sin over/under-fetching → menos latencia
- Esquema tipado y autodocumentado
- Agrega datos de varios servicios en una consulta
- Evoluciona sin versionar la URL

Costos a tener en cuenta

- Caching HTTP más complejo que REST
- Problema N+1 en resolvers (cuidar consultas)
- Control de complejidad de queries (límites)
- Curva de aprendizaje del esquema
- Puede ser demasiado para una API simple

GraphQL en MLOps



Metadatos de experimentos

Consultar runs, params y métricas de MLflow con una sola query flexible.



Agregación de servicios

Combinar en una respuesta datos del registry, del feature store y del linaje.



Linaje navegable

Recorrer el grafo dato → feature → modelo → despliegue desde el cliente.

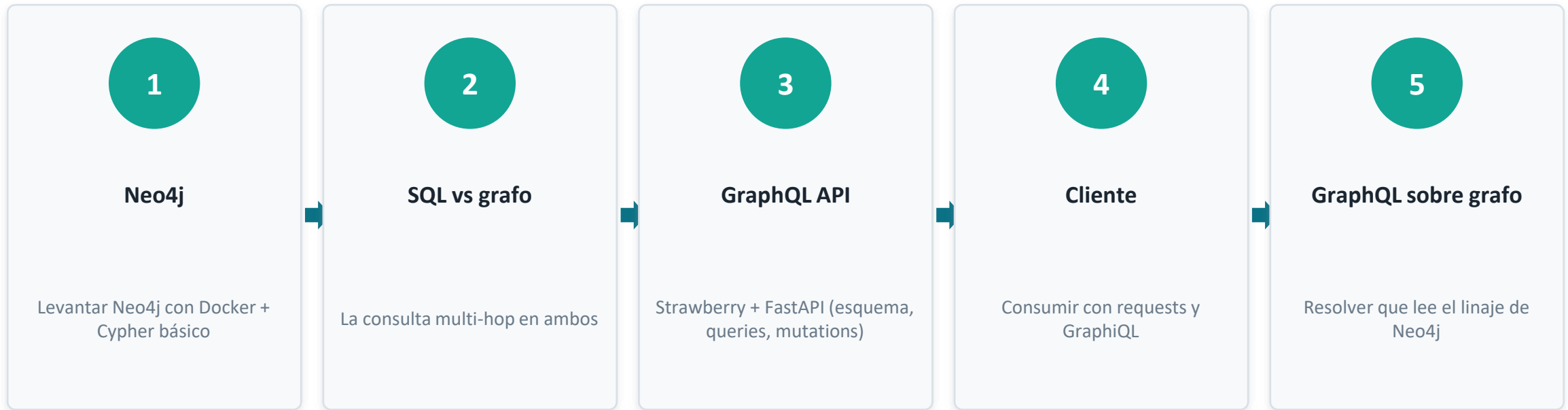


Un contrato para muchos

Frontend, equipos y dashboards piden cada uno lo que necesitan del mismo esquema.

Qué construimos

Dos piezas que se combinan: una base de grafos con el linaje y una API GraphQL para consultarla. Todo con uv y Docker; el código está en clase2/Practica del repo.



Reusa y depura los notebooks `neo4j_sql`, `GraphQL` y `GraphQL_REST_API`, migrados a uv.

Neo4j con Docker + Cypher

```
# 1) levantar Neo4j
docker run --name neo4j-tp \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/testpass neo4j:latest

// 2) crear nodos y relación (Cypher)
CREATE (a:Person {name:'Alice'})
CREATE (m:Movie {title:'Movie A'})
CREATE (a)-[:ACTED_IN]->(m);

// 3) consultar
MATCH (p:Person)-[:ACTED_IN]->(m) RETURN p,m;
```

Qué mirar

- UI del navegador en localhost:7474
- Driver Python (bolt) en :7687
- El patrón (a)-[:REL]->(b) es el corazón
- Las relaciones tienen tipo y dirección

NOTEBOOK neo4j_sql.ipynb

La misma pregunta, dos motores

```
# SQLite en memoria (relacional)
import sqlite3
conn = sqlite3.connect(':memory:')
# ... CREATE TABLE Persons/Movies/Relationships
# co-actores de Bob => 4 JOINS

# Neo4j (grafo)
from neo4j import GraphDatabase
drv = GraphDatabase.driver('bolt://localhost:7687',
                          auth=('neo4j', 'testpass'))
# co-actores de Bob => un patrón MATCH
```

NOTEBOOK neo4j_sql.ipynb

La conclusión

A más saltos entre relaciones, la consulta SQL se vuelve verbosa (muchos JOINS) mientras el patrón de grafo se mantiene legible y suele ser más rápido.

GraphQL con FastAPI + Strawberry

```
import strawberry
from strawberry.fastapi import GraphQLRouter
from fastapi import FastAPI

@strawberry.type
class Book:
    id: str; title: str; author: str

@strawberry.type
class Query:
    @strawberry.field
    def books(self) -> list[Book]: ...

schema = strawberry.Schema(query=Query)
app = FastAPI(); app.include_router(
    GraphQLRouter(schema), prefix='/graphql')
```

Qué mirar

- El esquema es el contrato tipado
- GraphQL queda en /graphql
- Cada campo lo produce un resolver
- `uv add strawberry-graphql fastapi uvicorn`

NOTEBOOK GraphQL.ipynb · docker_tutorial.ipynb

Queries: pide solo lo que necesitas

```
# solo título y autor
query {
  books { title author }
}

# filtrar por id (argumento)
query {
  book(id: "b2") { title year }
}
```

La idea central

La MISMA API responde formas distintas según lo que pida el cliente: un dashboard pide dos campos, un reporte pide todos, sin tocar el servidor. Los argumentos (id) filtran.

NOTEBOOK GraphQL.ipynb

Mutations: crear y borrar

```
mutation AddBook($t:String!, $a:String!, $y:Int){  
  createBook(data:{title:$t, author:$a, year:$y}){  
    id title author year  
  }  
}  
  
mutation { deleteBook(id:"b1"){ id title } }
```

Qué mirar

- Las mutations escriben (CRUD)
- Variables tipadas (\$t, \$a, \$y)
- Devuelves los campos que quieras del resultado
- Mismo endpoint que las queries

NOTEBOOK GraphQL.ipynb

Cliente Python + GraphQL

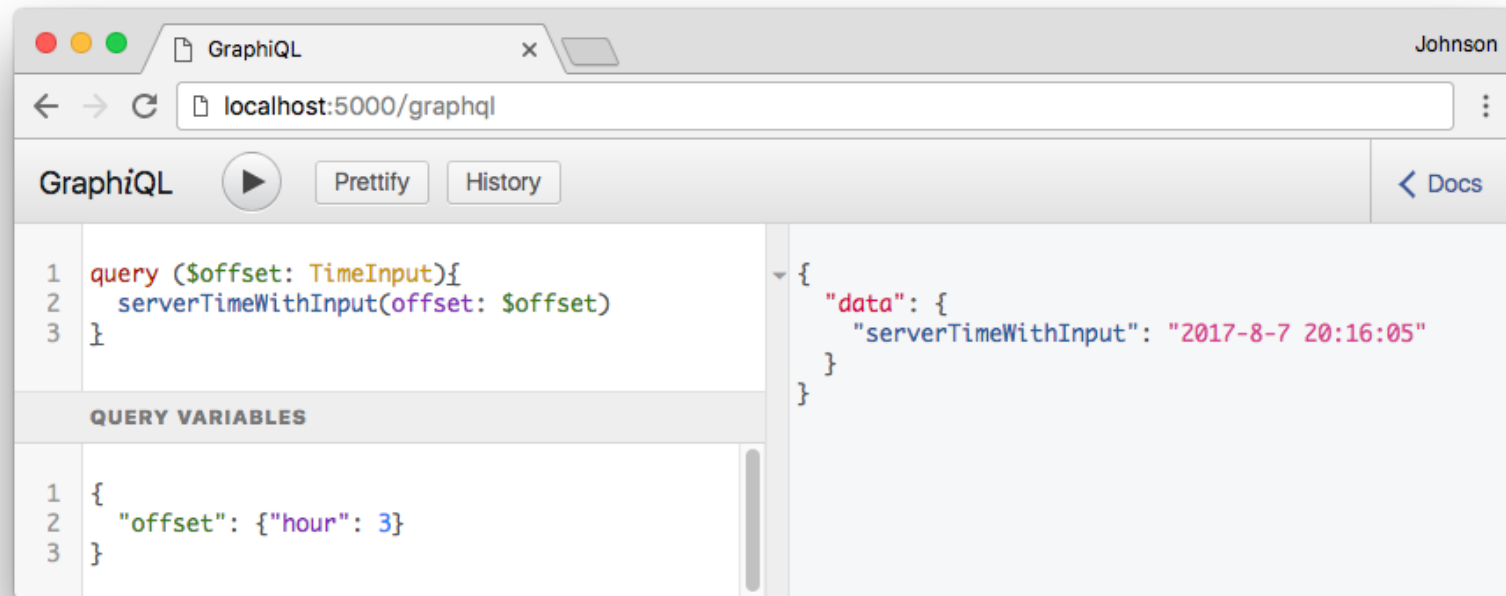
```
import requests
URL='http://localhost:8000/graphql'
q='query { books { title author } }'
r=requests.post(URL, json={'query': q})
print(r.json()['data']['books'])
# variables: json={'query':..., 'variables':{...}}
```

Dos formas de probar

- GraphQL en el navegador (/graphql)
- Cliente Python con requests
- Un POST con { query, variables }
- Explorar el esquema en 'Docs'

NOTEBOOK GraphQL.ipynb · rest_vs_graphql.ipynb

GraphQL: la query y sus variables



GraphQL en /graphql: a la izquierda la query, abajo las variables, a la derecha la respuesta con exactamente los campos pedidos.

El resolver de GraphQL consulta Neo4j: exponemos el linaje del modelo por una query flexible.

```
@strawberry.type
class Query:
    @strawberry.field
    def lineage(self, model: str) -> list[str]:
        cy = '''MATCH (d:Dataset)-[:DERIVES*]->
                (:Model {name:$m}) RETURN d.name'''
        with driver.session() as ses:
            rows = ses.run(cy, m=model)
            return [r['d.name'] for r in rows]
```

Qué logramos

- GraphQL = la puerta flexible.
- Neo4j = el motor de relaciones.
- El cliente pregunta el linaje de un modelo con una sola query.
- Es el patrón de metadatos de la plataforma.

NOTEBOOK graphql_neo4j_lineage.ipynb

Errores comunes (y cómo evitarlos)



Rango de Python muy amplio

Strawberry pide 3.11+: fija python>=3.11 en el proyecto.



Olvidar levantar Neo4j

El driver falla si el contenedor no está corriendo (:7687).



Credenciales/host hardcodeados

Usa variables de entorno (.env), no strings en el código.



Problema N+1 en resolvers

Cuida que un resolver no dispare una consulta por cada item.



Traer de más en la query

Pide solo los campos necesarios: es la gracia de GraphQL.



Notebook suelto

El código va en el repo, ejecutable de punta a punta.



Consigna

- Define un esquema GraphQL (Strawberry) para tu modelo: tipo Model con name, version y metrics.
- Query que devuelva las métricas/experimentos (de MLflow local o simulados).
- Pruébalo desde GraphiQL y desde un cliente Python.
- Compara la MISMA lectura contra tu endpoint REST de la Sesión 1.

Se evalúa

- Corre de punta a punta
- El esquema tipa entrada y salida
- La query pide solo lo necesario
- Reflexión REST vs GraphQL en el README

Actividad: `mini_tp2_actividad.ipynb` (starter).
Opcional (+): resolver que lea el linaje desde Neo4j.

Dónde quedó la plataforma

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

Hoy logramos

- Consultamos metadatos con GraphQL
- Modelamos el linaje como grafo (Neo4j)
- Sabemos cuándo GraphQL gana a REST



Próxima · Sesión 3

gRPC: cuando dos servicios internos necesitan hablar con latencia mínima y un contrato estricto, ni REST ni GraphQL son la mejor herramienta.

Para la próxima: ten tu Mini-TP 2 corriendo.

Referencias y recursos

Herramientas y código

- Strawberry GraphQL — strawberry.rocks · FastAPI — fastapi.tiangolo.com
- Neo4j — neo4j.com/docs · Cypher — neo4j.com/docs/cypher-manual · driver: `pip/uv add neo4j`
- Repositorio del curso: `clase2/Practica` (`neo4j_sql`, `GraphQL`, `GraphQL_REST_API`, `python-graphql-docker-tutorial`).

Lectura

- GraphQL — Introduction (graphql.org/learn).
- Designing Machine Learning Systems — Chip Huyen (O'Reilly): metadata y model store.